TRƯỜNG ĐẠI HỌC THỦY LỢI

KHOA CÔNG NGHỆ THÔNG TIN

GIÁO TRÌNH

THỰC HÀNH PHÁT TRIỂN ỨNG DỤNG CHO THIẾT BỊ DI ĐỘNG

Hà Nội, 2.2025

MỤC LỤC

CHƯƠNG 1. Làm quen 3

Bài 1) Tạo ứng dụng đầu tiên 3

1.1) Android Studio và Hello World 3

1.2) Giao diện người dùng tương tác đầu tiên 34

1.3) Trình chỉnh sửa bố cục 42

1.4) Văn bản và các chế độ cuộn 42

1.5) Tài nguyên có sẵn 42

Bài 2) Activities 42

2.1) Activity và Intent 42

2.2) Vòng đời của Activity và trạng thái 42

2.3) Intent ngầm định 42

Bài 3) Kiểm thử, gỡ lỗi và sử dụng thư viện hỗ trợ 42

3.1) Trình gỡ lỗi 42

3.2) Kiểm thử đơn vị 42

3.3) Thư viện hỗ trợ 42

CHƯƠNG 2. Trải nghiệm người dùng 43

Bài 1) Tương tác người dùng 43

1.1) Hình ảnh có thể chọn 43

1.2) Các điều khiển nhập liệu 43

1.3) Menu và bộ chọn 43

1.4) Điều hướng người dùng 43

1.5) RecycleView 43

Bài 2) Trải nghiệm người dùng thú vị 43

2.1) Hình vẽ, định kiểu và chủ đề 43

2.2) Thẻ và màu sắc 43

2.3) Bố cục thích ứng 43
Bài 3) Kiểm thử giao diện người dùng 43
3.1) Espresso cho việc kiểm tra UI 43
CHƯƠNG 3. Làm việc trong nền 43
Bài 1) Các tác vụ nền 43
1.1) AsyncTask 43
1.2) AsyncTask và AsyncTaskLoader 43
1.3) Broadcast receivers 43
Bài 2) Kích hoạt, lập lịch và tối ưu hóa nhiệm vụ nền 43
2.1) Thông báo 43
2.2) Trình quản lý cảnh báo 43
2.3) JobScheduler 43
CHƯƠNG 4. Lưu dữ liệu người dùng 44
Bài 1) Tùy chọn và cài đặt 44
1.1) Shared preferences 44
1.2) Cài đặt ứng dụng 44
Bài 2) Lưu trữ dữ liệu với Room 44
2.1) Room, LiveData và ViewModel 44
2.2) Room, LiveData và ViewModel 44

CHƯƠNG 1. LÀM QUEN

Bài 1) Tạo ứng dụng đầu tiên

1.1) Android Studio và Hello World

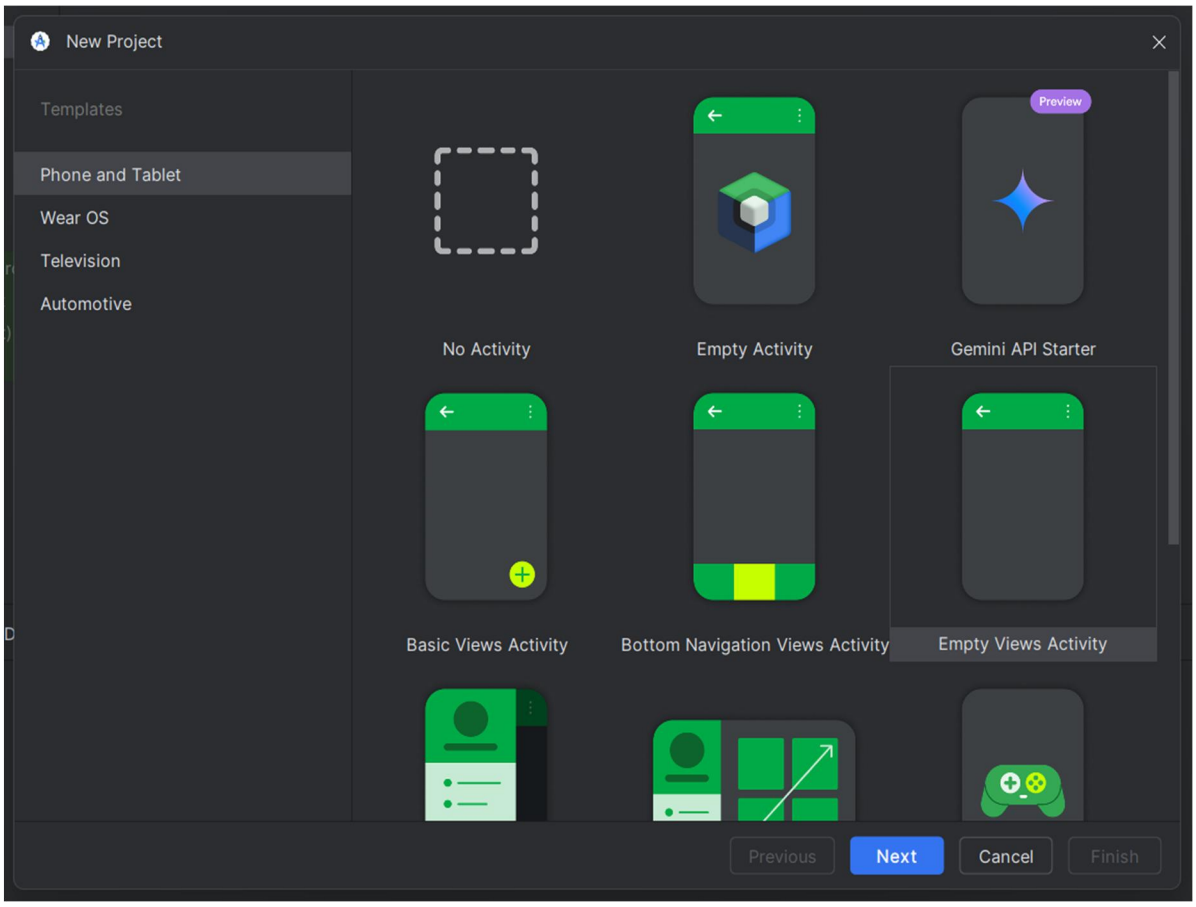
Giới thiệu

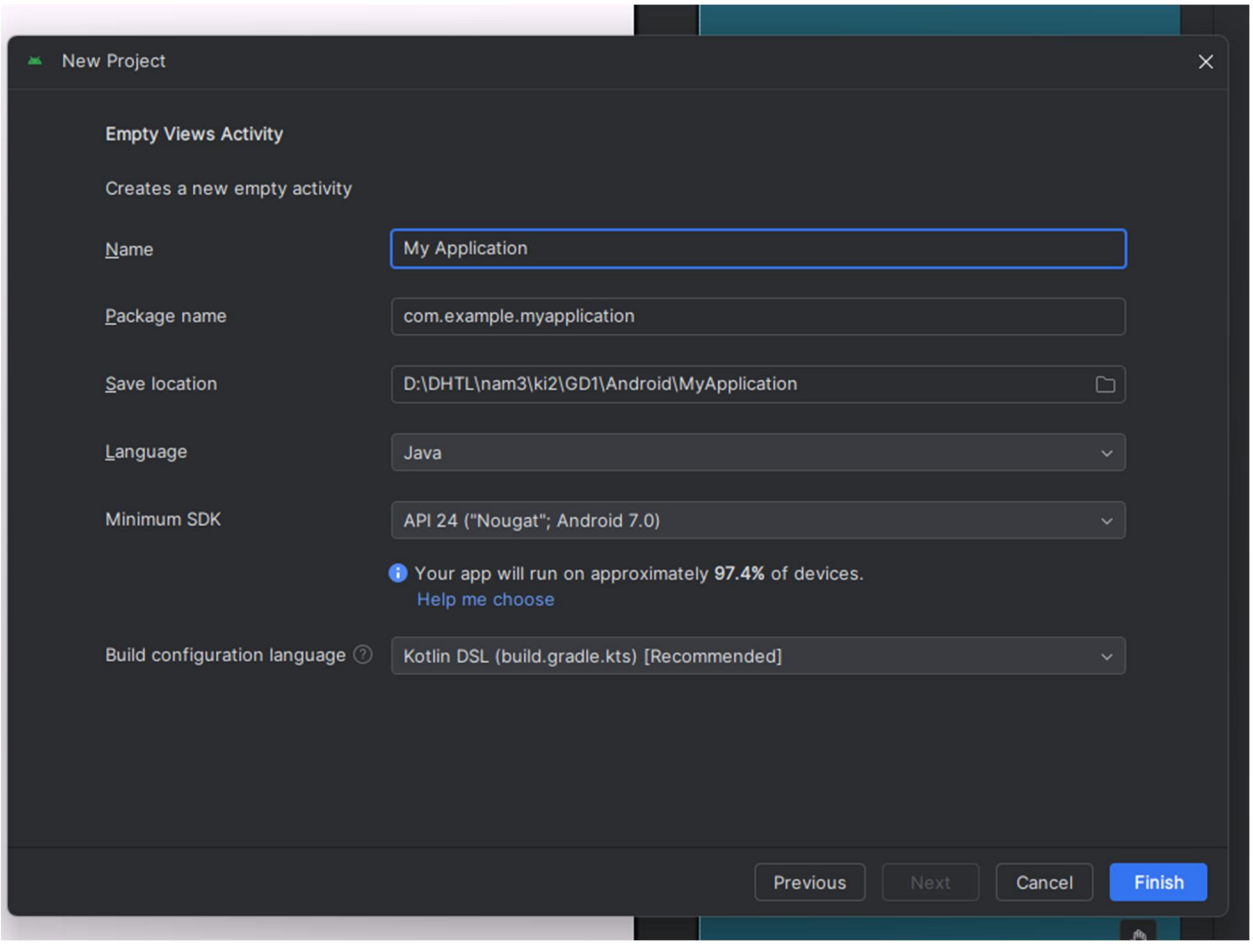
Trong bài thực hành này, bạn sẽ tìm hiểu cách cài đặt Android Studio, môi trường phát triển Android. Bạn cũng sẽ tạo và chạy ứng dụng Android đầu tiên của mình, Hello World, trên một trình giả lập và trên một thiết bị vật lý.

Những gì Bạn nên biết

Bạn nên có khả năng:

- Hiểu quy trình phát triển phần mềm tổng quát cho các ứng dụng lập trình hướng đối tượng sử dụng một IDE (môi trường phát triển tích hợp) như Android Studio.
- Chứng minh rằng bạn có ít nhất 1-3 năm kinh nghiệm trong lập trình hướng đối tượng, với một phần trong số đó tập trung vào ngôn ngữ lập trình Java. (Các bài thực hành này sẽ không giải thích về lập trình hướng đối tượng hoặc ngôn ngữ Java.





Những gì Bạn sẽ cần:

- Một máy tính chạy Windows hoặc Linux, hoặc một Mac chạy macOS. Xem trang tải xuống Android Studio để biết yêu cầu hệ thống cập nhật.
- Truy cập Internet hoặc một phương pháp thay thế để tải các cài đặt mới nhất của Android Studio và Java lên máy tính của bạn.

Những gì bạn sẽ học

- Cách cài đặt và sử dụng IDE Android Studio.
- Cách sử dụng quy trình phát triển để xây dựng ứng dụng Android.
- Cách tạo một dự án Android từ một mẫu.
- Cách thêm thông điệp ghi lại vào ứng dụng của bạn để phục vụ mục đích gỡ lỗi.

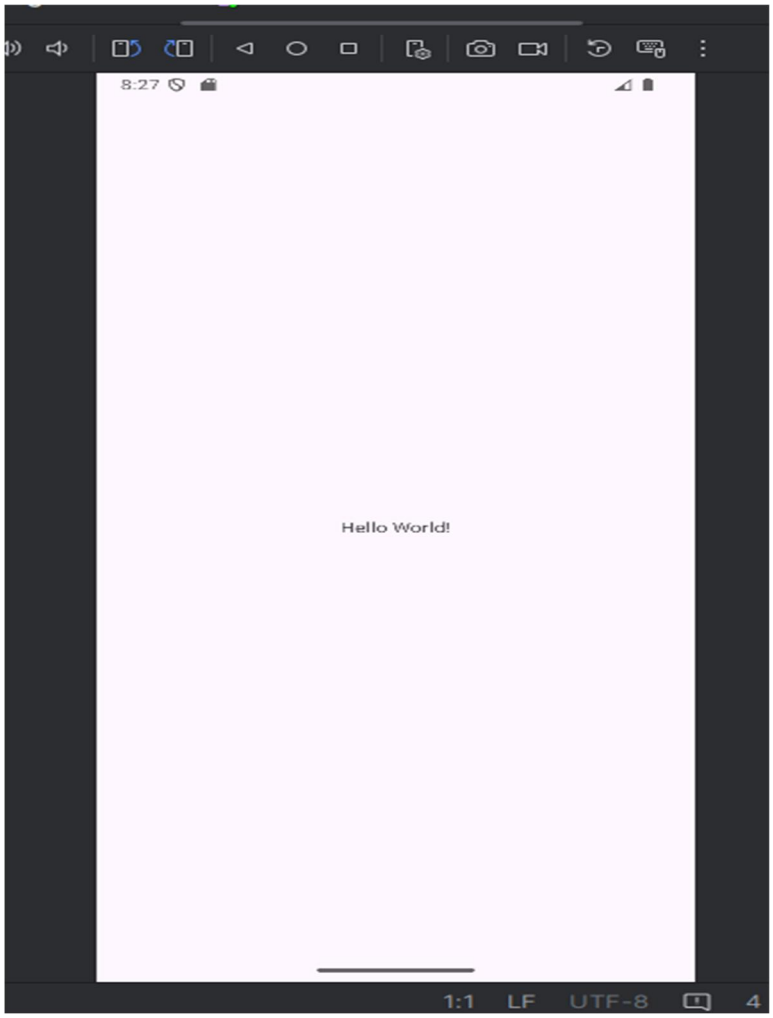
Những gì bạn sẽ làm

- Cài đặt môi trường phát triển **Android Studio**.
- Tạo một trình giả lập (thiết bị ảo) để chạy ứng dụng của bạn trên máy tính.
- Tạo và chạy ứng dụng **Hello World** trên các thiết bị ảo và vật lý.
- Khám phá cấu trúc dự án.
- Tạo và xem các thông điệp ghi lại từ ứng dụng của bạn.
- Khám phá tệp **AndroidManifest.xml**

Tổng quan về ứng dụng

Sau khi hoàn tất cài đặt **Android Studio,** bạn sẽ tạo từ mẫu, 1 dự án mới cho ứng dụng **Hello World.** Đây là 1 dự án ứng dụng đơn giản hiển thị chuỗi *"Hello World"* trên màn hình của máy ảo thiết bị Android hoặc thiết bị vật lý.

Dưới đây là giao diện của ứng dụng sau khi đã hoàn thành:



Nhiệm vụ 1: Cài đặt Android Studio

Android Studio cung cấp một môi trường phát triển tích hợp hoàn chỉnh (IDE), bao gồm một trình soạn thảo mã tiên tiến và bộ mẫu ứng dụng đa dạng. Ngoài ra, nó còn chứa các công cụ hỗ trợ phát triển, gỡ lỗi, kiểm thử và tối ưu hiệu suất, giúp quá trình phát triển ứng dụng trở nên nhanh chóng và dễ dàng hơn. Bạn có thể kiểm thử ứng dụng của bạn trên nhiều trình giả lập được cấu hình sẵn hoặc trực tiếp trên thiết bị di động của mình, xây dựng ứng dụng hoàn chỉnh và phát hành chúng trên cửa hàng Google Play.

Chú ý: Android Studio luôn dược cải tiến liên tục. Để cập nhật thông tin chi tiết về yêu cầu cấu hình hệ thống cũng như hướng dẫn cài đặt, vui lòng tham khảo trang chủ của Android Studio.

Android Studio có thể dùng cho các máy tính chạy hệ điều hành Windows hoặc Linux, và cho các máy Macs chạy macOS. Phiên bản OpenJDK (Java Development Kit) mới nhất được tích hợp sẵn trong Android Studio.

Để bắt đầu sử dụng Android Studio, trước tiên hãy kiểm tra các yêu cầu hệ thống để đảm bảo rằng máy tính của bạn đáp ứng đủ các tiêu chí để sử dụng. Quá trình cài đặt tương tự cho tất cả các nền tảng. Kcác khác biệt nếu có sẽ được ghi chú bên dưới.

1. Truy cập trang Android Developers và làm theo hướng dẫn để tải về và cài đặt Android Studio.
2. Chấp nhận các cấu hình mặc định cho tất cả các bước cài đặt, và đảm bảo rằng tất cả các thành phần cần thiết đều được chọn.
3. Sau khi cài đặt xong, Setup Wizard sẽ tự động tải về và cài đặt một số thành phần bổ sung, bao gồm Android SDK. Hãy kiên nhẫn, quá trình này có thể mất chút thời gian tùy thuộc vào tốc độ Internet của bạn, và một số bước có thể trông có vẻ thừa thãi.
4. Khi quá trình tải về hoàn tất, Android Studio sẽ khởi động và bạn đã sẵn sàng để tạo dự án đầu tiên của mình.

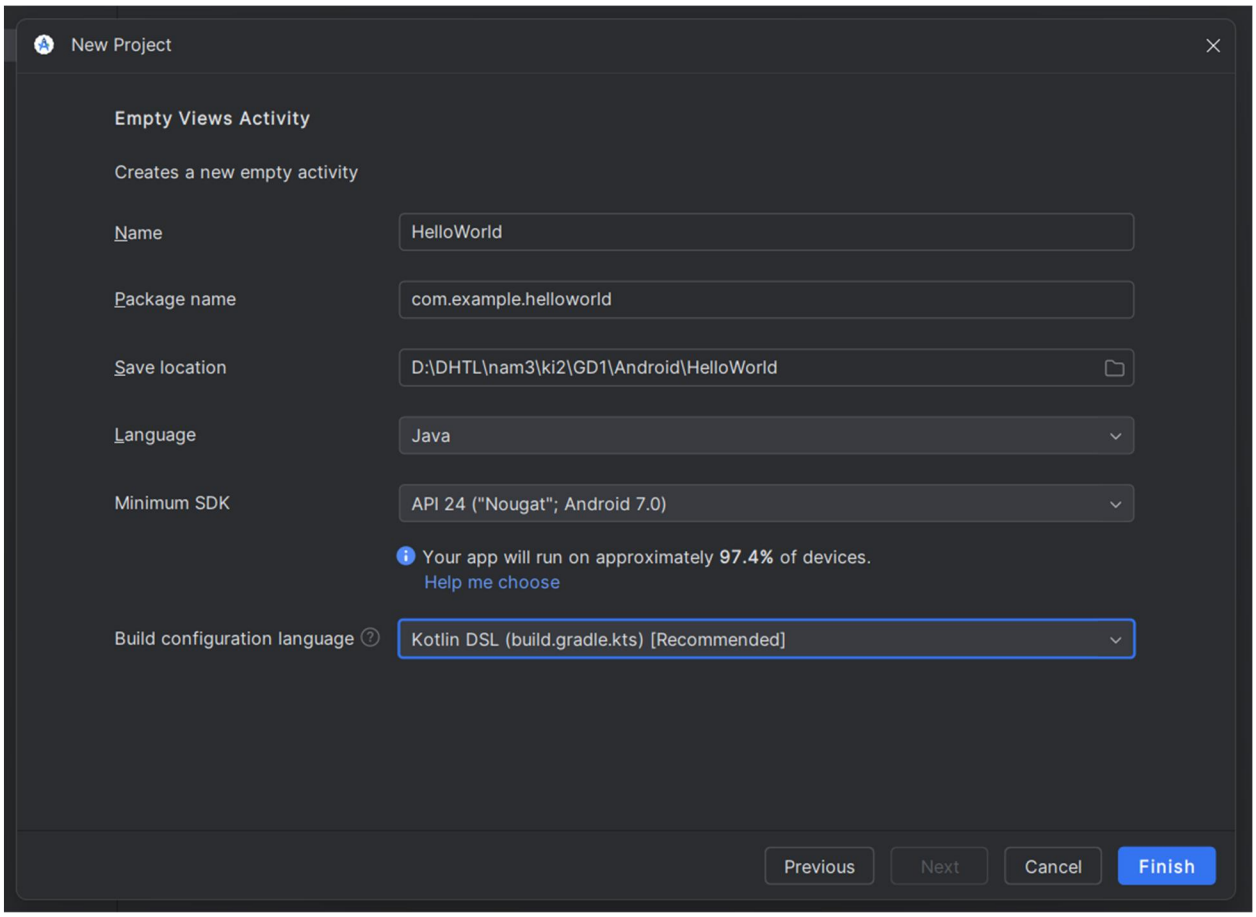
Khắc phục sự cố: Nếu bạn gặp vấn đề với quá trình cài đặt, hãy xem phần ghi chú phát hành của Android Studio hoặc tìm sự hỗ trợ từ người hướng dẫn của bạn.

Nhiệm vụ 2: Tạo 1 ứng dụng Hello World

Ở nhiệm vụ này, bạn sẽ tạo 1 ứng dung và hiển thị ” Hello World” nhằm xác nhận rằng Android Studio đã được cài đặt đúng cách, đồng thời làm quen với những kiến thức cơ bản về phát triển ứng dụng với Android Studio.

2.1. Tạo dự án

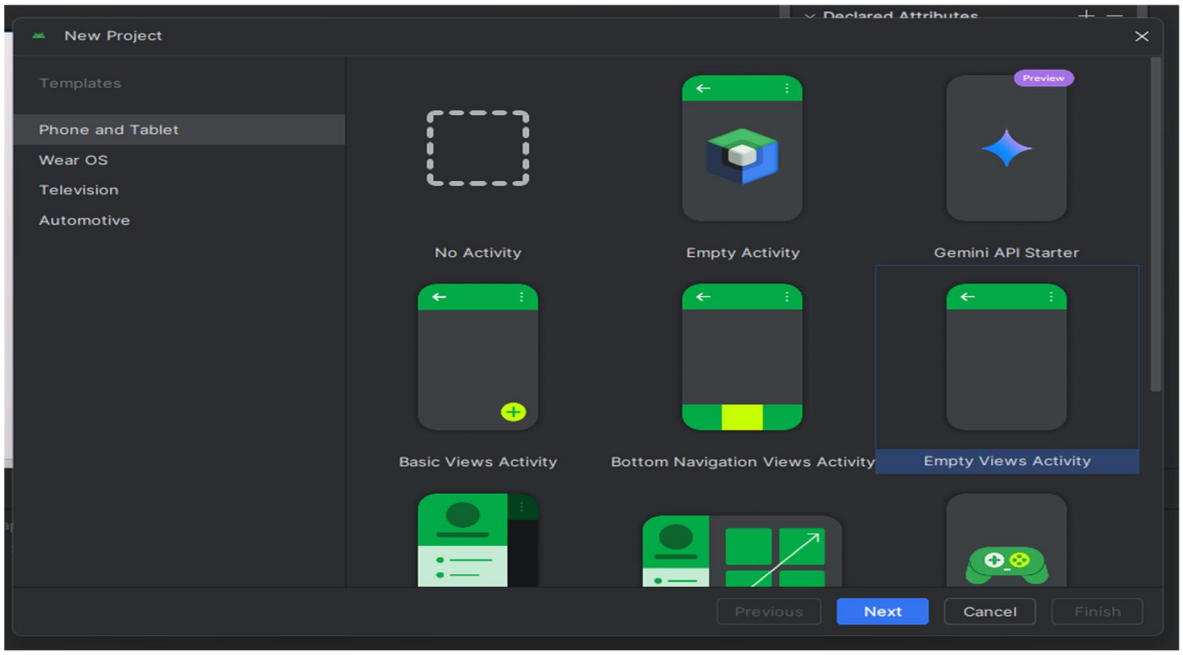
1. Mở Android Studio nếu nó chưa được mở
2. Trong màn hình chính hiển thị **Welcome to Android Studio**, hãy nhấp vào **Start a new Android Studio project**.
3. Trong cửa sổ **Create Android Project**, nhập **Hello World** vào trường “**Name**".



4. Xác nhận rằng vị trí lưu trữ dự án (**Save location**) mặc định là nơi bạn muốn lưu trữ ứng dụng Hello World cũng như các dự án Android Studio khác, hoặc thay đổi thành thư mục mà bạn ưu tiên.
5. Xác nhận mặc định “android.example.com” cho trường Company Domain, hoặc tạo 1 tên miền khác riêng biệt.

Nếu bạn không có ý định phát hành ứng dụng, có thể chấp nhận giá trị mặc định. Lưu ý rằng thay đổi tên gói của ứng dụng của bạn sau này sẽ làm tăng khối lượng công việc

6. Không chọn tùy chọn **Include C++ support** và **Include Kotlin support**, sau đó nhấn **Next**.
7. Ở màn hình **Target Android Devices**, đảm bảo rằng lựa chọn **Phone and Tablet** đã được chọn. Kiểm tra xem Minimum SDK đã được đặt là **API 24: Android 7.0 Nougat**; nếu chưa, hãy sử dụng menu popup để cài đặt đúng.
   Với cách thiết lập này được sử dụng bởi các ví dụ trong bài học của khóa học này. Tính đến thời điểm hiện tại, các thiết lập này giúp ứng dụng Hello World của bạn tương thích với 97% các thiết bị Android đang hoạt động trên Google Play Store.
8. Bỏ chọn tùy chọn **Include Instant App support** và tất cả các tùy chọn khác. Sau đó nhấn **Next**. Nếu dự án của bạn yêu cầu thêm các thành phần bổ sung cho target SDK mà bạn đã chọn, Android Studio sẽ tự động cài đặt chúng.\
9. Cửa sổ **Add an Activity** xuất hiện. Một Activity là một tác vụ đơn lẻ, tập trung mà người dùng có thể thực hiện và là thành phần thiết yếu của bất kỳ ứng dụng Android nào. Một Activity thường có kèm theo một bố cục (layout) xác định cách các thành phần giao diện người dùng xuất hiện trên màn hình. Android Studio cung cấp các mẫu Activity để giúp bạn bắt đầu. Đối với dự án Hello World, hãy chọn **Empty Activity** như hình minh họa bên dưới, và nhấn **Next**.



10. Màn hình **Configure Activity** xuất hiện (nó sẽ khác nhau tùy theo template bạn đã chọn ở bước trước). Theo mặc định, Activity rỗng được cung cấp bởi template có tên là **MainActivity**. Bạn có thể thay đổi tên này nếu muốn, nhưng bài học này sử dụng **MainActivity**.
11. Hãy đảm bảo rằng tùy chọn **Generate Layout file** được đánh dấu. Tên layout mặc định là **activity_main**. Bạn có thể thay đổi điều này nếu muốn, nhưng bài học này sử dụng **activity_main**.
12. Đảm bảo rằng tùy chọn **Backwards Compatibility (App Compat)** được đánh dấu. Điều này giúp ứng dụng của bạn tương thích ngược với các phiên bản Android trước đó.
13. Nhấn Finish.

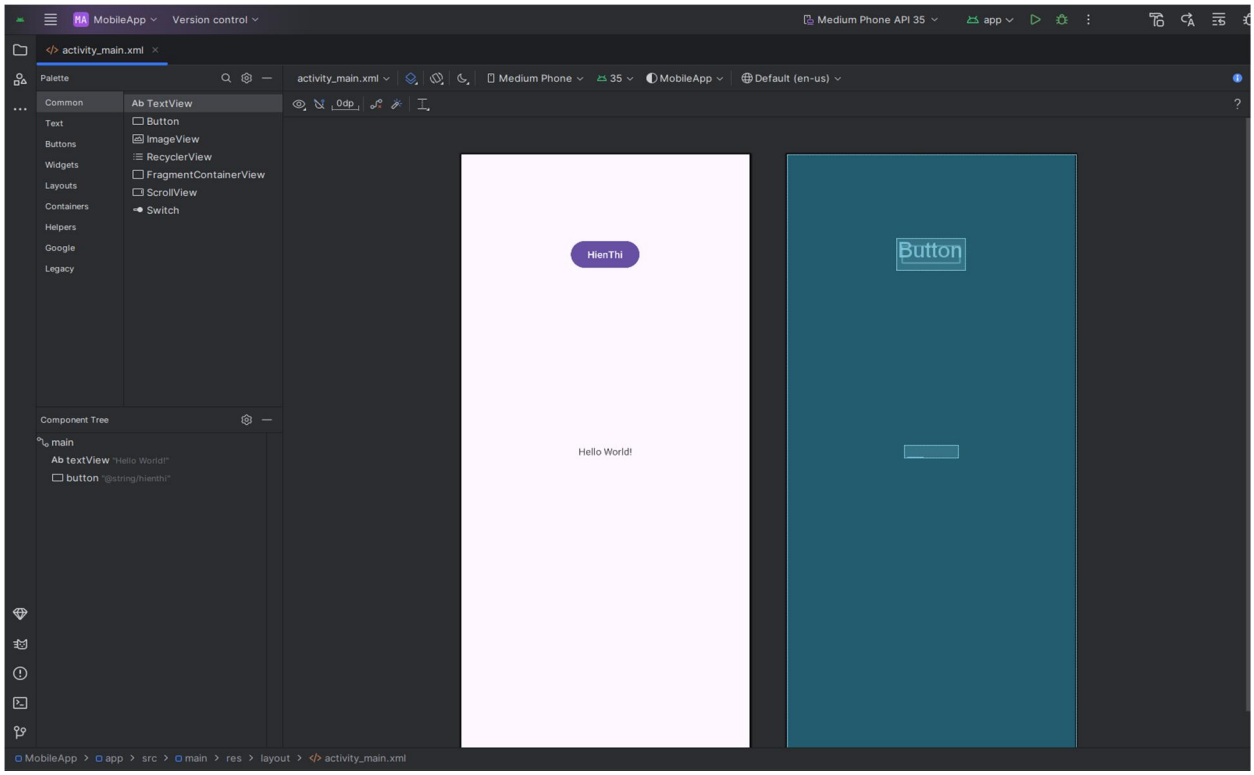
Android Studio sẽ tạo 1 thư mục chứa dự án của bạn, và xây dựng dự án sử dụng Gradie(quá trình này có thể mất một vài phút)

Mẹo: Tham khảo trang Configure your build developer của nhà phát triển để biết thêm thông tin chi tiết.

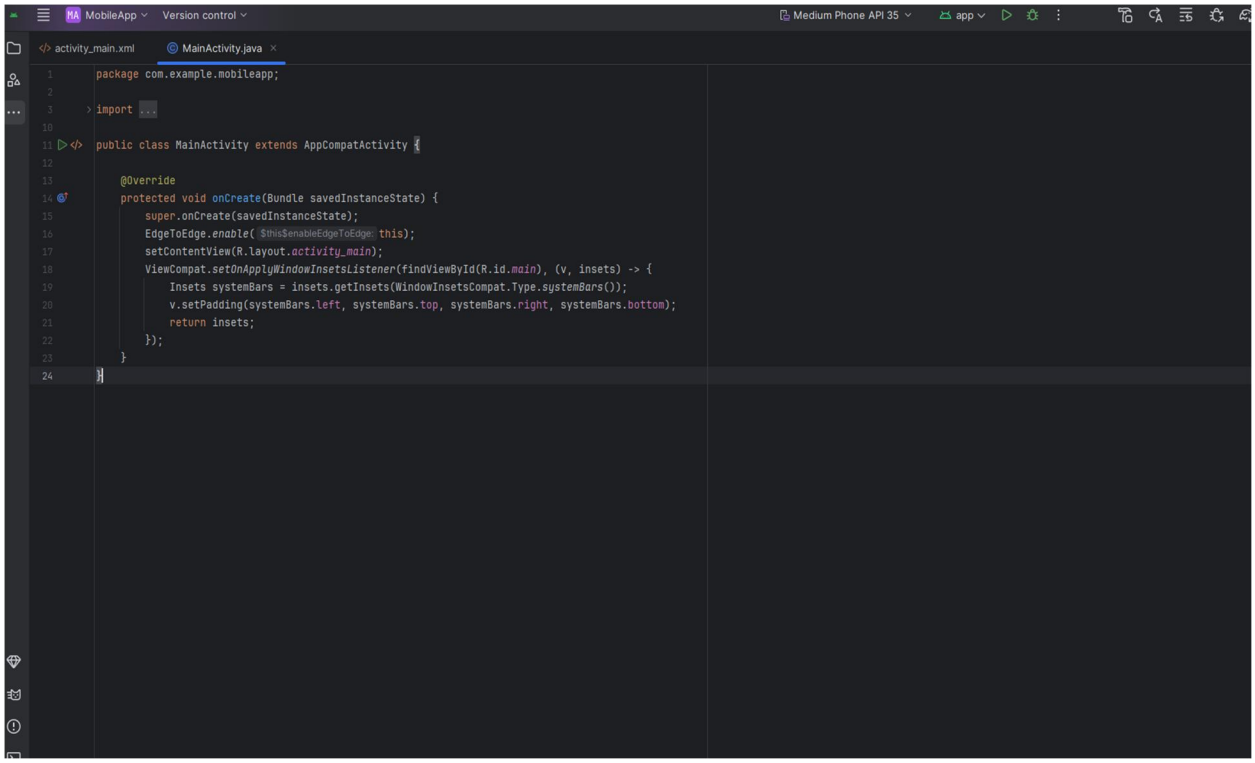
Bạn cũng có thể thấy một thông báo "Tip of the day" chứa các phím tắt và các mẹo hữu ích khác. Nhấn **Close** để đóng thông báo.

Trình chỉnh sửa soạn thảo của Android Studio xuất hiện. Thực hiện các bước sau:

1. Nhấp vào tab **activity_main.xml** để xem trình chỉnh sửa bố cục.
2. Nhấp vào tab **Design** trong trình chỉnh sửa bố cục (nếu chưa được chọn) để hiển thị phiên bản đồ họa của bố cục như hình minh họa bên dưới.



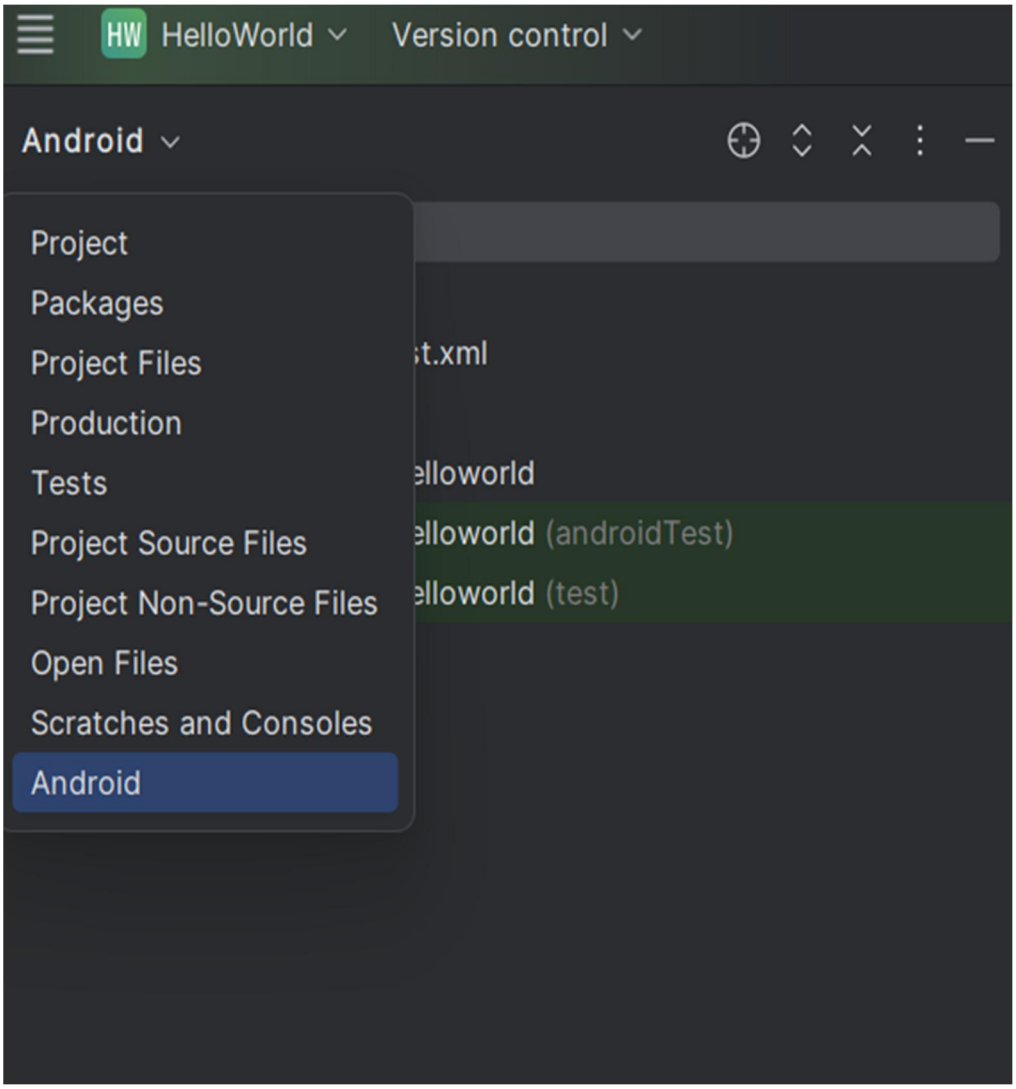
3. Chọn thẻ **MainActivity.java** để xem trình soạn thảo mã như được hiển thị bên dưới.



2.2. Khám phá Dự án > Bảng điều khiển Android

Trong bài thực hành này, bạn sẽ khám phá cách dự án được tổ chức trong Android Studio.

1. Nếu chưa được chọn, hãy nhấp vào tab **Project** ở cột tab dọc bên trái của cửa sổ Android Studio. Cửa sổ **Project** sẽ xuất hiện.
2. Để xem dự án theo cấu trúc tiêu chuẩn của Android, hãy chọn **Android** từ menu thả xuống ở phía trên của cửa sổ **Project**, như hình minh họa bên dưới.

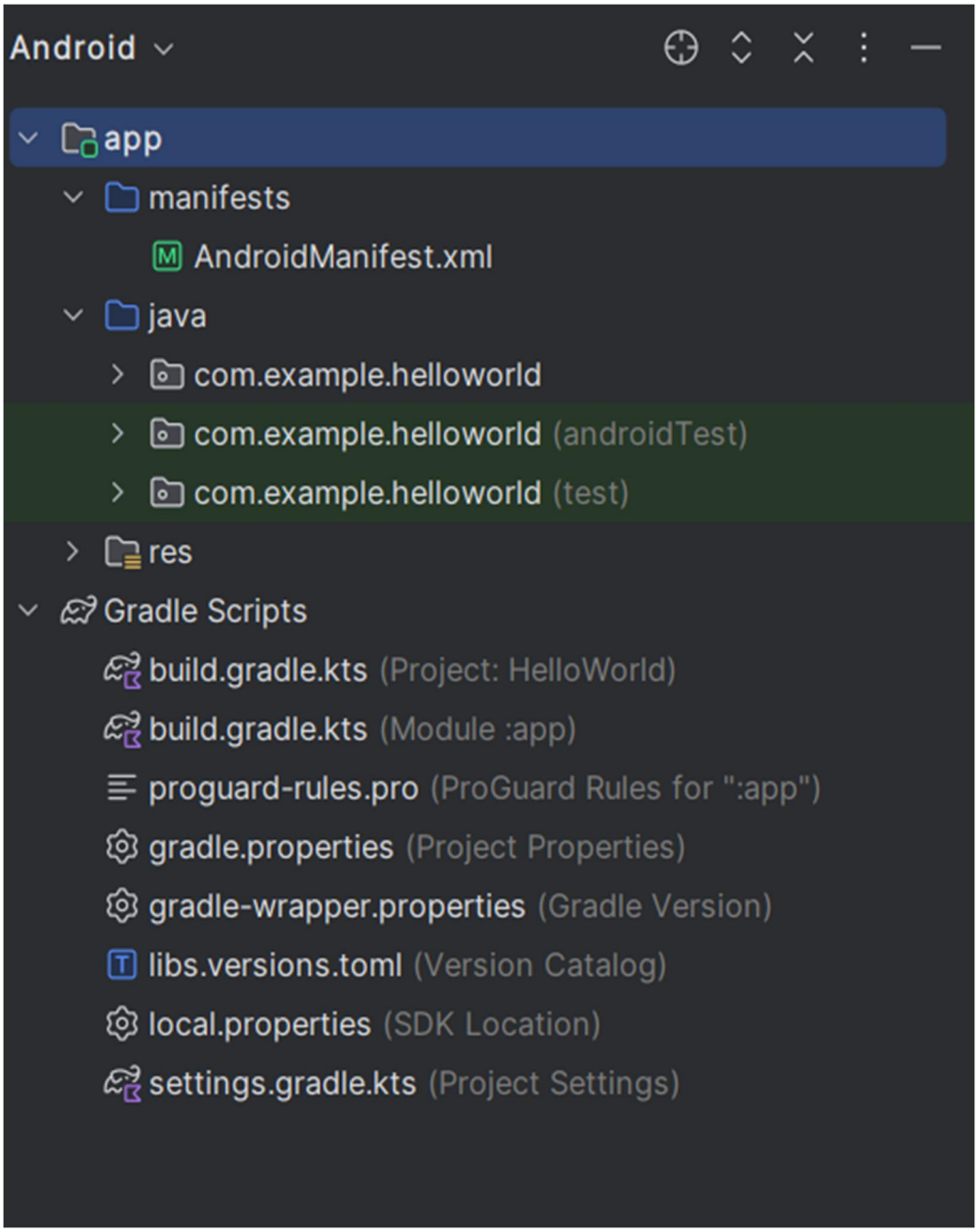


Chú ý: Chương này và các chương khác đề cập đến cửa sổ dự án, khi được đặt ở chế độ Android như là **Project** > **Android**.

2.3. Khám phá thư mục Gradle Scripts

thống xây dựng Gradle trong Android Studio giúp bạn dễ dàng tích hợp các tệp nhị phân bên ngoài hoặc các module thư viện khác vào quá trình xây dựng dưới dạng các phụ thuộc.

Khi bạn vừa tạo một dự án ứng dụng, cửa sổ **Project > Android** sẽ hiển thị với thư mục **Gradle Scripts** được mở rộng như hình minh họa bên dưới.

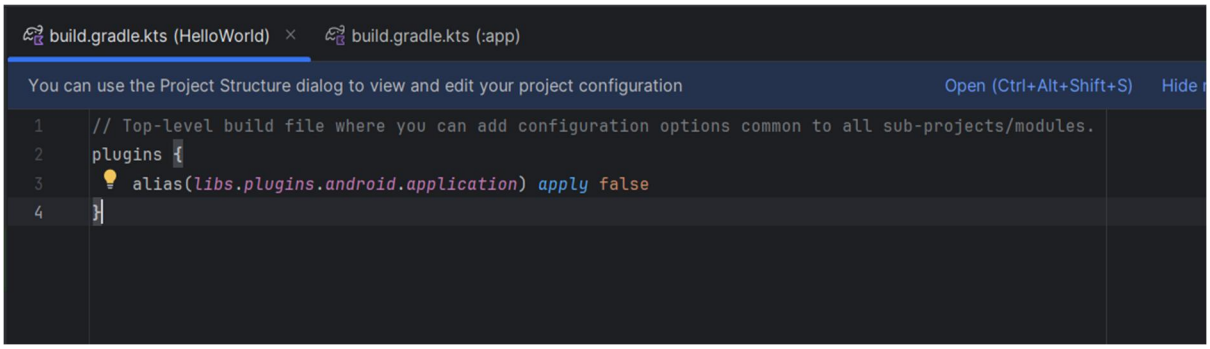


Thực hiện các bước sau để tìm hiểu về hệ thống **Gradle Scripts:**

1. Nếu thư mục **Gradle Scripts** chưa được mở, hãy nhấp vào biểu tượng hình tam giác để mở rộng nó. Thư mục này chứa tất cả các tệp cần thiết cho hệ thống build.
2. Nhìn vào tệp **build.gradle (Project: HelloWorld)**:

Đây là nơi bạn sẽ tìm thấy các tùy chọn cấu hình chung cho tất cả các module cấu thành dự án của bạn. Mỗi dự án Android Studio đều có một tệp build.gradle cấp cao. Hầu hết thời gian, bạn sẽ không cần chỉnh sửa bất kỳ gì tệp này, nhưng việc hiểu rõ nội dung của nó vẫn rất hữu ích.

Mặc định, tệp build cấp cao sử dụng khối **buildscript** để định nghĩa các kho lưu trữ và các phụ thuộc chung cho tất cả các module trong dự án. Khi phụ thuộc của bạn không phải là một thư viện nội bộ hay một cây tập tin cục bộ, Gradle sẽ tìm các tệp cần thiết ở các kho lưu trữ trực tuyến được chỉ định trong khối **repositories** của tệp này. Mặc định, các dự án Android Studio mới khai báo **JCenter** và **Google** (bao gồm cả Google Maven repository) làm các vị trí kho lưu trữ:



3. Nhìn vào tệp **build.gradle(Module:app):**
   Bên cạnh tệp **build.gradle** cấp dự án, mỗi mô-đun đều có tệp build.gradle riêng, cho phép bạn định cấu hình cài đặt bản dựng cho từng mô-đun cụ thể (ứng dụng HelloWorld chỉ có một module duy nhất). Việc cấu hình các thiết lập build này giúp bạn cung cấp các tùy chọn đóng gói tùy chỉnh, chẳng hạn như thêm các kiểu xây dựng và các sản phẩm đi kèm. Bạn cũng có thể ghi đè các thiết lập có trong tệp **AndroidManifest.xml** hoặc tệp **build.gradle** cấp dự án. Tệp này thường là tệp bạn sẽ chỉnh sửa khi thay đổi các cấu hình cấp ứng dụng, chẳng hạn như khai báo các phụ thuộc trong phần **dependencies**. Bạn có thể khai báo một phụ thuộc thư viện bằng cách sử dụng một trong các cấu hình

phụ thuộc khác nhau; mỗi cấu hình phụ thuộc cung cấp cho Gradle các hướng dẫn khác nhau về cách sử dụng thư viện. Ví dụ, câu lệnh

implementation fileTree(dir: 'libs', include: ['*.jar'])

thêm 1 phụ thuộc cho tất cả các tệp ".jar" ở trong thư mục libs.

Dưới đây là tệp **build.gradle (Module: app)** cho ứng dụng HelloWorld:

build.gradle.kts (HelloWorld) × build.gradle.kts (:app) ×

You can use the Project Structure dialog to view and edit your project configuration Open (Ctrl+Alt+Shift+S) Hide notification

```
1  plugins {
2      alias(libs.plugins.android.application)
3  }
4
5  android {
6      namespace = "com.example.helloworld"
7      compileSdk = 34
8
9      defaultConfig {
10         applicationId = "com.example.helloworld"
11         minSdk = 24
12         targetSdk = 34
13         versionCode = 1
14         versionName = "1.0"
15
16         testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
17     }
18
19     buildTypes {
20         release {
21             isMinifyEnabled = false
22             proguardFiles(
23                 getDefaultProguardFile( name: "proguard-android-optimize.txt"),
24                 "proguard-rules.pro"
25             )
26         }
27     }
28     compileOptions {
29         sourceCompatibility = JavaVersion.VERSION_11
30         targetCompatibility = JavaVersion.VERSION_11
```

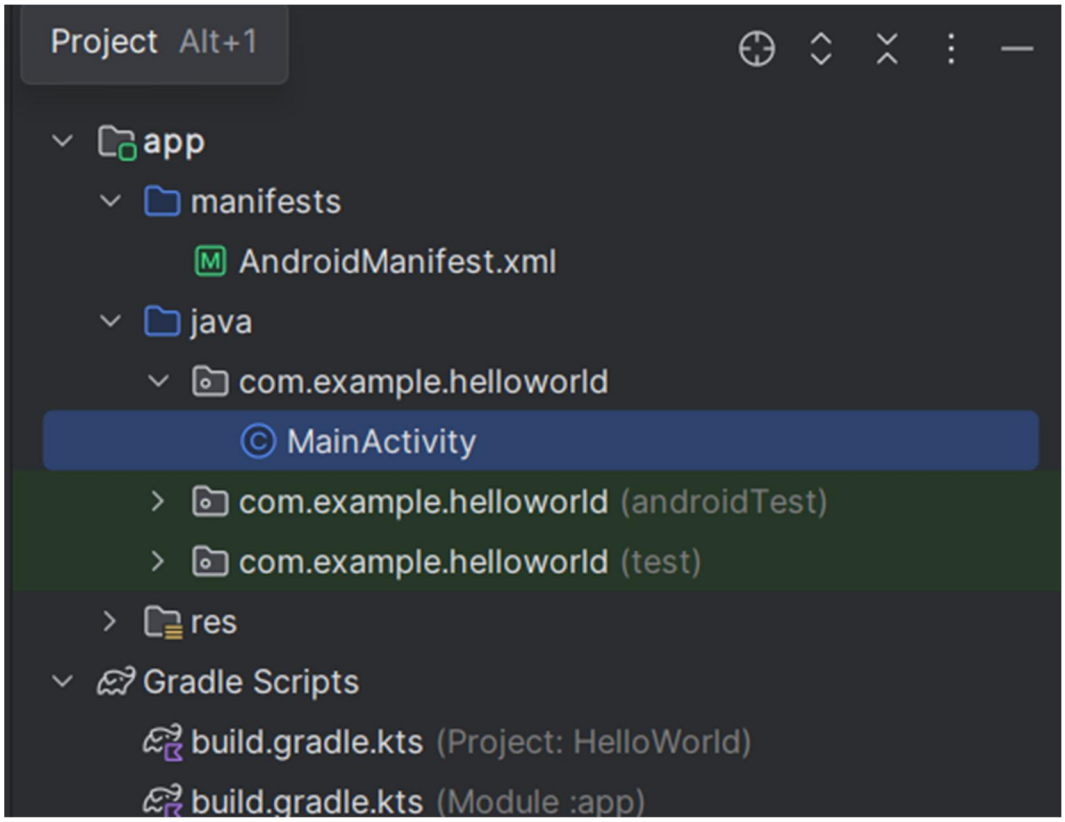
```
27     }
28     compileOptions {
29         sourceCompatibility = JavaVersion.VERSION_11
30         targetCompatibility = JavaVersion.VERSION_11
31     }
32 }
33
34 dependencies {
35
36     implementation(libs.appcompat)
37     implementation(libs.material)
38     implementation(libs.activity)
39     implementation(libs.constraintlayout)
40     testImplementation(libs.junit)
41     androidTestImplementation(libs.ext.junit)
42     androidTestImplementation(libs.espresso.core)
43 }
```

4. Nhấp vào hình tam giác để đóng **Gradle Scripts.**

2.4. Khám phá các thư mục ứng dụng và thư mục res

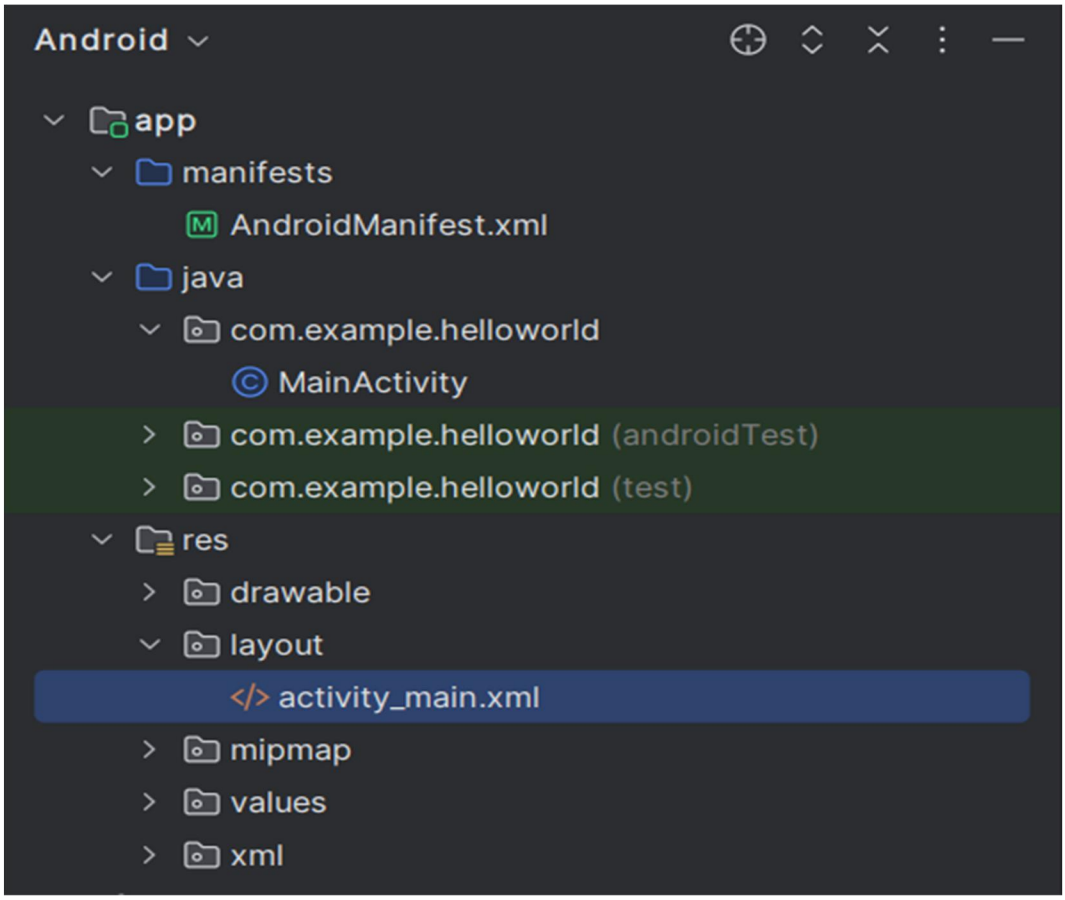
Tất cả code và tài nguyên cho ứng dụng đều nằm ở thư mục ứng dụng và thư mục res

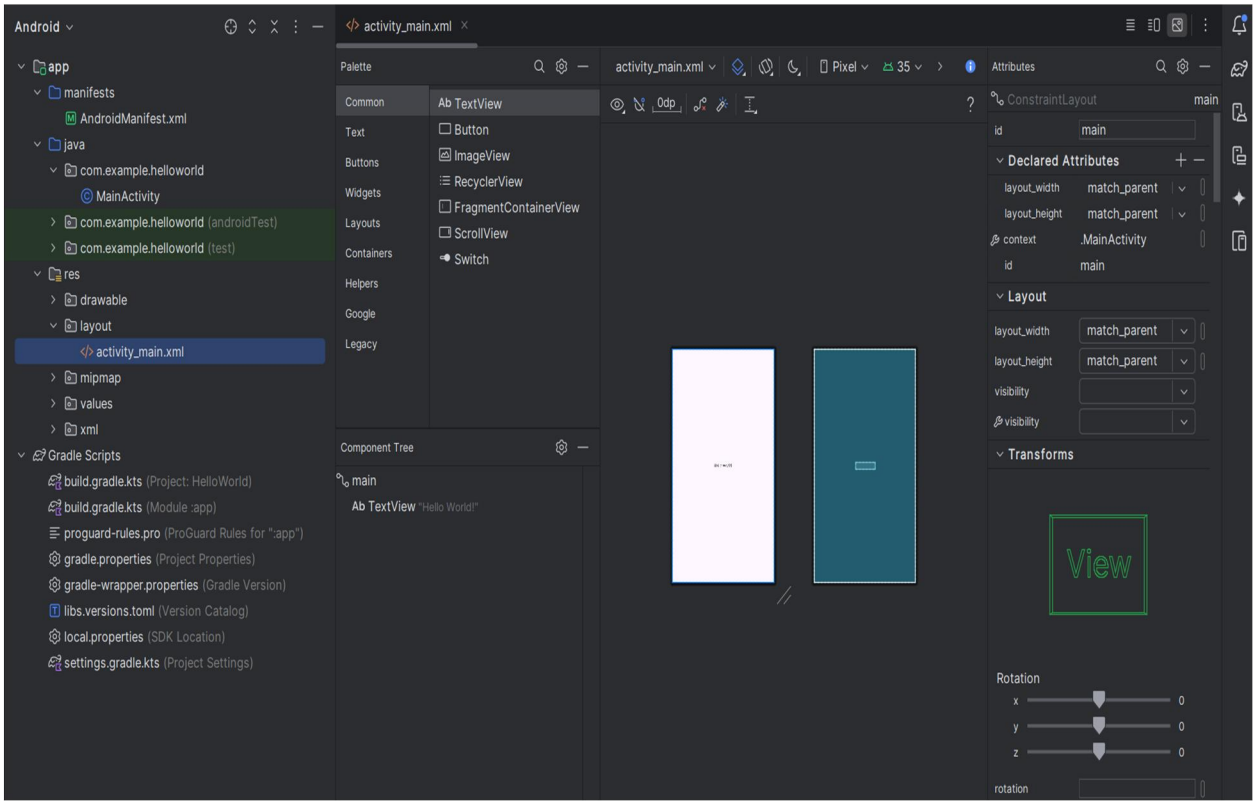
1. Mở thư mục app, sau đó thư mục java, và cuối cùng là thư mục com.example.android.helloworld để thấy tệp MainActivity java. Nhấp đúp vào tệp này sẽ mở nó trong trình soạn thảo mã nguồn.



   Thư mục **java** chứa các tệp lớp Java được sắp xếp trong ba thư mục con, như hình minh họa ở trên. Thư mục **com.example.android.helloworld** (hoặc tên miền mà bạn đã chỉ định) chứa tất cả các tệp của một gói ứng dụng. Hai thư mục còn lại được sử dụng cho mục đích kiểm thử và sẽ được mô tả trong một bài học khác. Đối với ứng dụng Hello World, chỉ có một gói và nó chứa tệp **MainActivity.java**. Tên của Activity đầu tiên (màn hình) mà người dùng thấy, đồng thời khởi tạo các tài nguyên cho toàn bộ ứng dụng, thường được đặt là **MainActivity** (phần mở rộng tệp được bỏ qua trong cửa sổ **Project > Android**).

2. Mở thư mục res và sau đó mở thư mục layout, nháy đúp chuột và tệp **activity_main.xml** để mở nó trong trình chỉnh sửa bố cục.


Android
app
manifests
AndroidManifest.xml
java
com.example.helloworld
MainActivity
com.example.helloworld (androidTest)
com.example.helloworld (test)
res
drawable
layout
activity_main.xml
mipmap
values
xml


Android
app
manifests
AndroidManifest.xml
java
com.example.helloworld
MainActivity
com.example.helloworld (androidTest)
com.example.helloworld (test)
res
drawable
layout
activity_main.xml
mipmap
values
xml
Gradle Scripts
build.gradle.kts (Project: HelloWorld)
build.gradle.kts (Module :app)
proguard-rules.pro (ProGuard Rules for ":app")
gradle.properties (Project Properties)
gradle-wrapper.properties (Gradle Version)
libs.versions.toml (Version Catalog)
local.properties (SDK Location)
settings.gradle.kts (Project Settings)
activity_main.xml
Palette
Common
Text
Buttons
Widgets
Layouts
Containers
Helpers
Google
Legacy
Ab TextView
Button
ImageView
RecyclerView
FragmentContainerView
ScrollView
Switch
Component Tree
main
Ab TextView "Hello World!"
activity_main.xml
Pixel
35
0dp
Attributes
ConstraintLayout
main
id
main
Declared Attributes
layout_width
match_parent
layout_height
match_parent
context
.MainActivity
id
main
Layout
layout_width
match_parent
layout_height
match_parent
visibility
visibility
Transforms
View
Rotation
x
0
y
0
z
0
rotation

Thư mục **res** chứa các tài nguyên của ứng dụng, chẳng hạn như bố cục chuỗi ký tự và hình ảnh. Một **Activity** thường được liên kết với một bố cục giao diện người dùng được định nghĩa trong một tệp XML, và tệp này thường được đặt tên theo tên của **Activity** đó.

2.5. Khám phá thư mục manifests

Thư mục **manifests** chứa các tệp cung cấp thông tin thiết yếu về ứng dụng cho hệ thống Android – thông tin mà hệ thống cần có trước khi có thể chạy bất kỳ mã nào của ứng dụng.

1. Mở rộng thư mục **manifests**.
2. Mở tệp **AndroidManifest.xml**.

activity_main.xml | AndroidManifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="HelloWorld"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.HelloWorld"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Tệp **AndroidManifest.xml** mô tả tất cả các thành phần của ứng dụng Android của bạn. Mỗi thành phần của ứng dụng, chẳng hạn như từng **Activity**, đều phải được khai báo trong tệp XML này. Trong các bài học tiếp theo của khóa học, bạn sẽ chỉnh sửa tệp này để thêm các tính năng và cấp quyền truy cập cho các tính năng đó. Để tìm hiểu thêm, hãy xem mục App Manifest Overview.

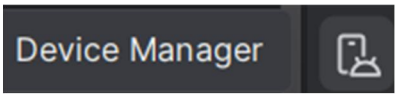
Nhiệm vụ 3: Sử dụng thiết bị ảo (trình giả lập)

Trong nhiệm vụ này, bạn sẽ sử dụng Android Virtual Device (AVD) manager để tạo một thiết bị ảo (còn được gọi là trình giả lập) mô phỏng cấu hình của một loại thiết bị Android cụ thể, và sử dụng thiết bị ảo đó để chạy ứng dụng. Lưu ý rằng **Android Emulator** có các yêu cầu bổ sung ngoài các yêu cầu hệ thống cơ bản của Android Studio.

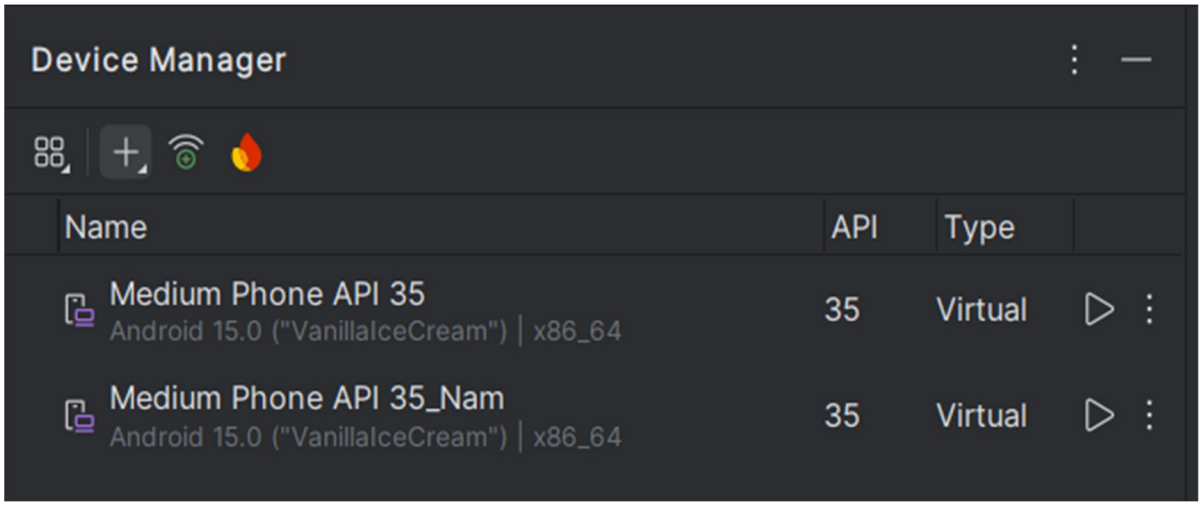
Khi sử dụng Trình quản lý AVD, bạn sẽ xác định các đặc điểm phần cứng của thiết bị, cấp độ API, bộ nhớ, giao diện và các thuộc tính khác của thiết bị và lưu dưới dạng thiết bị ảo. Với các thiết bị ảo, bạn có thể kiểm tra ứng dụng trên các cấu hình thiết bị khác nhau (như máy tính bảng và điện thoại) với các cấp độ API khác nhau mà không cần phải sử dụng thiết bị vật lý.

3.1. Tạo 1 thiết bị Andriod ảo (AVD)

Để chạy emulator trên máy tính của bạn, bạn cần tạo một cấu hình mô tả thiết bị ảo.

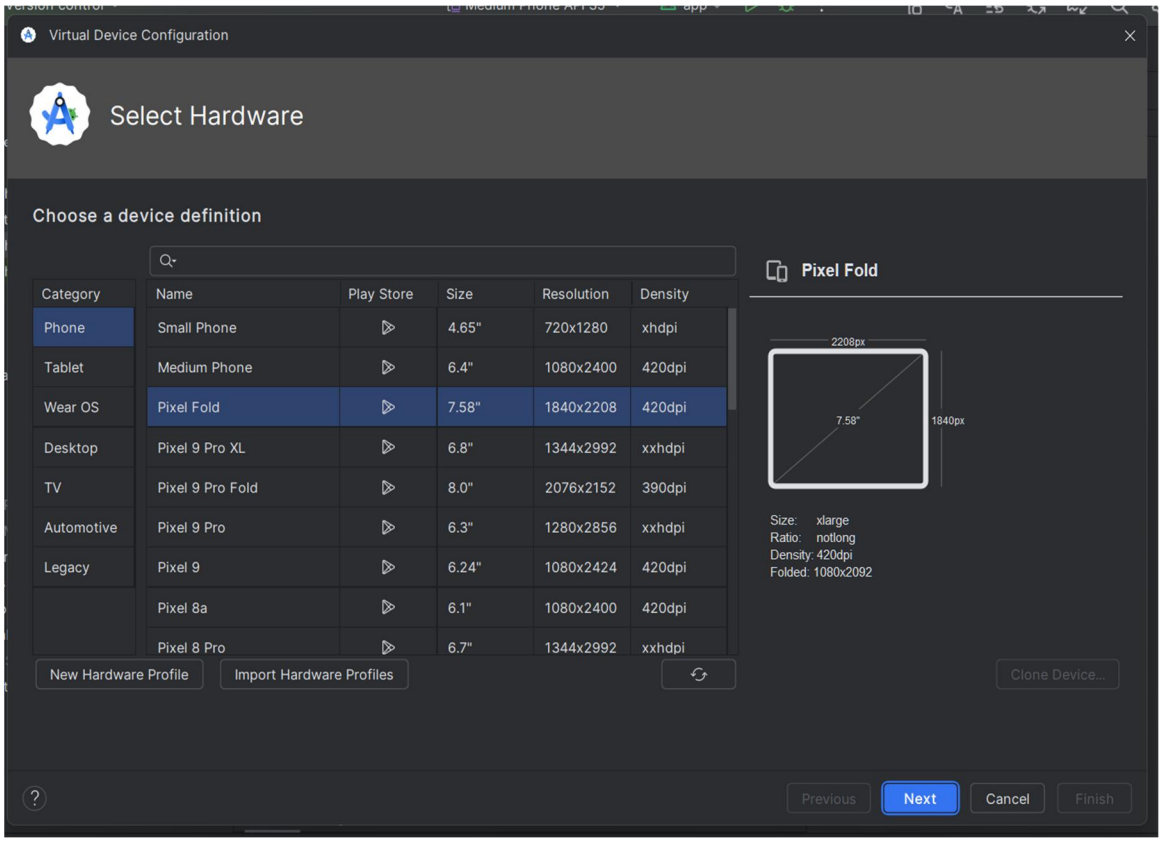
1. Ở trong **Android Studio**, chọn: **Tools** > **Android** > **AVD Manager,** hoặc nhấp vào biểu tượng Device Manager 

trên thanh công cụ. Màn hình **Device Manager** xuất hiện. Nếu bạn đã tạo thiết bị ảo trước đó, danh sách sẽ hiển thị chúng. Nếu chưa có, danh sách sẽ trống.

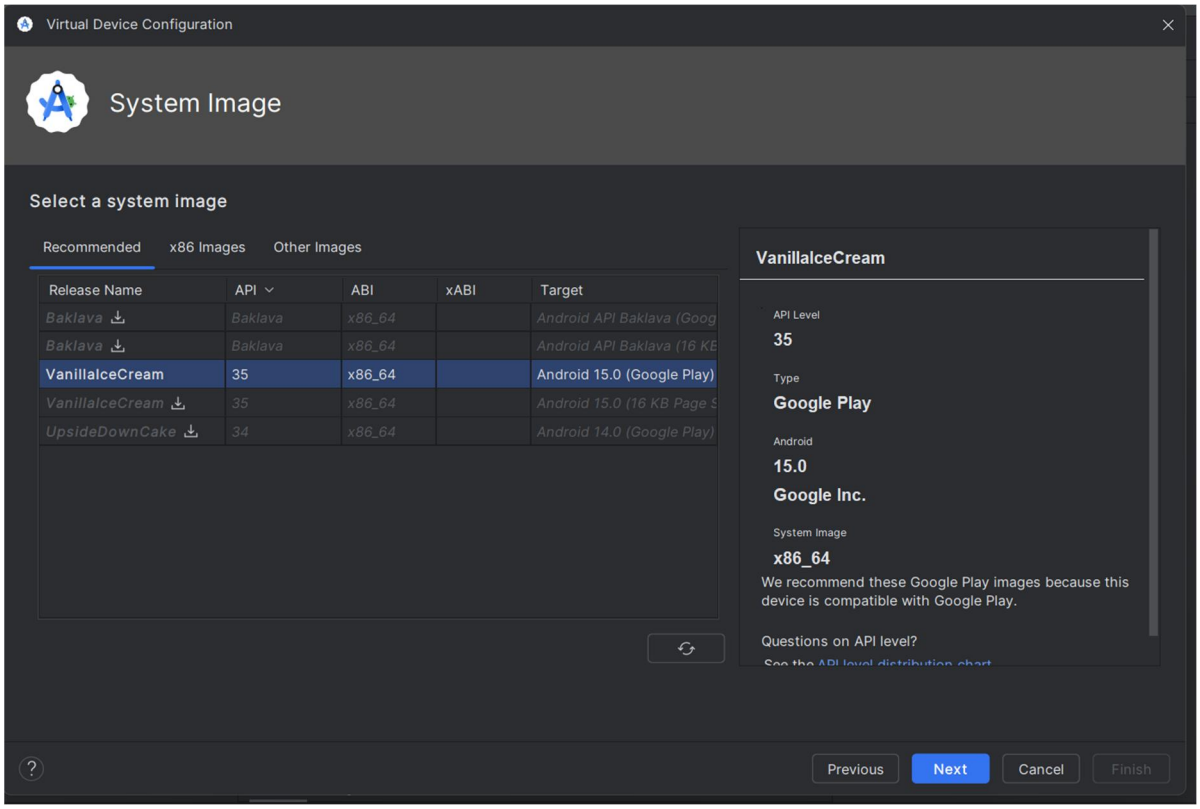


2. Nhấp vào **+Create Virtual Device**. Cửa sổ Select Hardware sẽ xuất hiện, hiển thị danh sách các thiết bị phần cứng được cấu hình sẵn. Đối với mỗi thiết bị,

bảng cung cấp một cột cho ích thước màn hình đường chéo (Size), độ phân giải màn hình tính bằng pixel (Resolution) và mật độ pixel(Density).



3. Chọn một thiết bị như **Pixel 9 Pro XL** hoặc **Pixel Fold** và nhấp vào **Next**. Màn hình ảnh hệ thống xuất hiện.
4. Nhấp vào thẻ **Recommended** nếu chưa chọn và chọn phiên bản hệ thống Android nào để chạy trên thiết bị ảo (chẳng hạn như **VanillalcaCream**).



Có nhiều phiên bản khả dụng hơn so với những phiên bản được hiển thị trong thẻ **Recommended**. Hãy xem các hệ điều hành **x86** và các hệ điều hành khác để xem chúng.

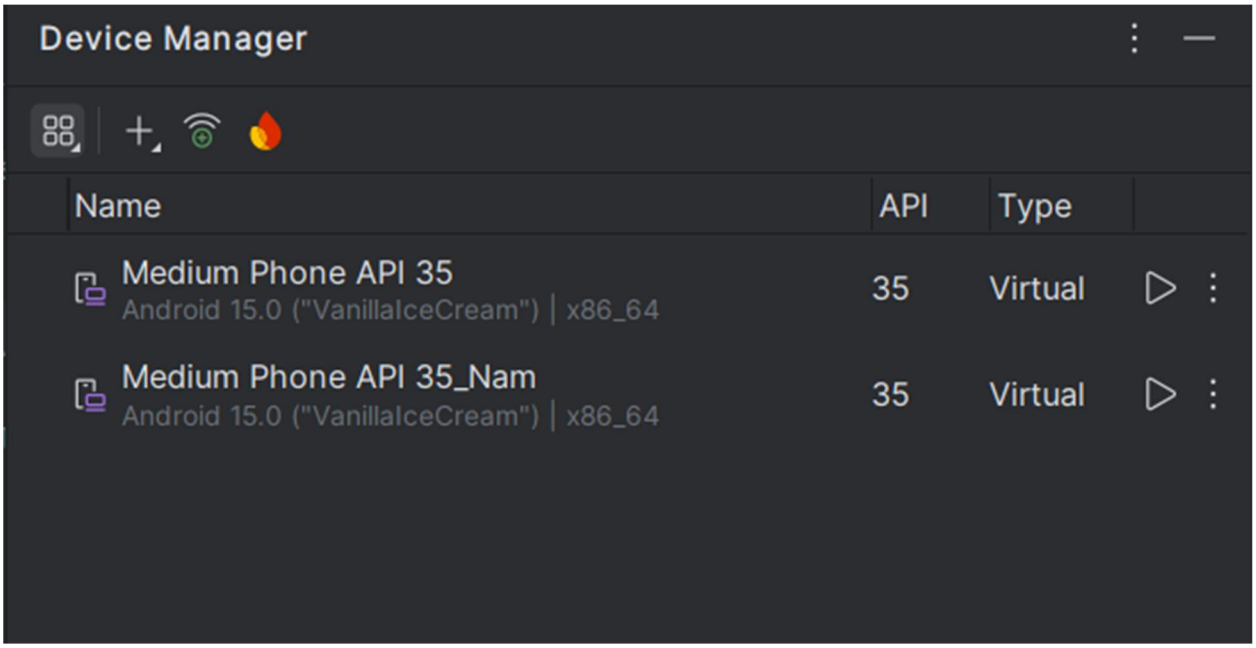
Nếu có liên kết **Download** bên cạnh một hệ điều hành bạn muốn sử dụng, điều đó có nghĩa là nó chưa được cài đặt. Nhấn vào liên kết để bắt đầu tải xuống, sau đó nhấn **Finish** khi quá trình tải hoàn tất.

5. Sau cùng chọn hệ điều hành, nhấn **Next**. Cửa sổ **Android Virtual Device (AVD)** sẽ xuất hiện. Bạn cũng có thể đổi tên thiết bị ảo (AVD). Kiểm tra lại cấu hình của bạn và nhấn **Finish**.

3.2. Chạy ứng dụng trên thiết bị ảo

Ở nhiệm vụ này, bạn sẽ hoàn tất chạy ứng dụng Hello World của mình.

6. Trong Android Studio, chọn **Run > Run app** hoặc nhấn vào biểu tượng **Run** trên thanh công cụ.
7. Trong cửa sổ **Select Deployment Target**, dưới mục **Available Virtual Devices**, chọn thiết bị ảo mà bạn vừa tạo và nhấn **OK**.



Trình giả lập sẽ khởi động giống như một thiết bị thật. Tùy vào tốc độ của máy tính, quá trình này có thể mất một chút thời gian. Ứng dụng của bạn được xây dựng và khi trình giả lập đã sẵn sàng, Android Studio sẽ tải ứng dụng lên trình giả lập và chạy ứng dụng.

Bạn sẽ thấy ứng dụng Hello World như trong hình như sau

Mẹo: Khi thử nghiệm trên thiết bị ảo, bạn nên khởi động thiết bị một lần, ngay từ đầu phiên của bạn. Bạn không nên đóng thiết bị cho đến khi hoàn tất thử nghiệm ứng dụng, để ứng dụng không phải trải qua quá trình khởi động thiết bị một lần nữa. Để đóng thiết bị ảo, hãy nhấp vào nút **X** ở đầu trình giả lập, chọn **Thoát** từ menu hoặc nhấn **Control-Q** trong Windows hoặc **Command-Q** trong macOS.

Nhiệm vụ 4: (Tùy chọn) Sử dụng thiết bị vật lý

Trong nhiệm vụ cuối cùng này, bạn sẽ chạy ứng dụng của mình trên thiết bị di động vật lý như điện thoại hoặc máy tính bảng. Bạn luôn phải kiểm tra ứng dụng của mình trên cả thiết bị ảo và vật lý.

Những gì bạn cần:

- Một thiết bị Android, chẳng hạn như điện thoại hoặc máy tính bảng.
- Cáp dữ liệu để kết nối thiết bị Android của bạn với máy tính qua cổng USB.
- Nếu bạn đang sử dụng hệ thống Linux hoặc Windows, có thể cần thực hiện thêm một số bước để chạy trên thiết bị phần cứng. Kiểm tra tài liệu Using Hardware Devices. Bạn cũng có thể cần cài đặt trình điều khiển USB phù hợp cho thiết bị của mình. Đối với trình điều khiển USB trên Windows, xem OEM USB Drivers.

4.1. Bật gỡ lỗi USB

Để Android Studio có thể giao tiếp với thiết bị của bạn, bạn phải bật **Gỡ lỗi USB** trên thiết bị Android. Tùy chọn này nằm trong **Cài đặt nhà phát triển** của thiết bị của bạn.

Trên Android 4.2 trở lên, màn hình **Cài đặt nhà phát triển** bị ẩn theo mặc định. Để hiển thị và bật **Gỡ lỗi USB**, hãy làm theo các bước sau:

1. Trên thiết bị của bạn, mở **Cài đặt**, tìm **Giới thiệu về điện thoại**, nhấn vào **Giới thiệu về điện thoại**, sau đó chạm vào **Số bản dựng bảy lần**.
2. Quay lại màn hình trước đó (**Cài đặt / Hệ thống**), bạn sẽ thấy mục **Tùy chọn nhà phát triển**. Nhấn vào **Tùy chọn nhà phát triển**.
3. Tìm và bật **Gỡ lỗi USB**.

4.2. Chạy ứng dụng ở trên thiết bị

Bây giờ bạn có thể kết nối thiết bị và chạy ứng dụng từ Android Studio.

1. Kết nối thiết bị của bạn với máy tính phát triển bằng cáp USB.
2. Nhấn vào nút **Run** trên thanh công cụ. Cửa sổ **Select Deployment Target** sẽ mở ra, hiển thị danh sách các trình giả lập và thiết bị đã kết nối.
3. Chọn thiết bị của bạn và nhấn **Ok.**

Android Studio sẽ cài đặt và chạy ứng dụng trên thiết bị của bạn.

Khắc phục sự cố

Nếu Android Studio không nhận diện được thiết bị của bạn, hãy thử các bước sau:

1. Rút và cắm lại thiết bị.
2. Khởi động lại Android Studio.

Nếu máy tính vẫn không tìm thấy thiết bị hoặc hiển thị là **"unauthorized" (chưa được ủy quyền)**, hãy làm theo các bước sau:

1. Rút thiết bị ra.
2. Trên thiết bị, mở **Tùy chọn nhà phát triển** trong **Cài đặt**.
3. Nhấn vào **Thu hồi quyền gỡ lỗi USB**.
4. Kết nối lại thiết bị với máy tính.
5. Khi có thông báo yêu cầu quyền, hãy cấp quyền.

Bạn có lẽ cần cài đặt trình điều khiển USB phù hợp cho thiết bị của mình. Xem tài liệu sử dụng thiết bị phần cứng

Nhiệm vụ 5: Thay đổi cấu hình Gradle của ứng dụng

Trong nhiệm vụ này, bạn sẽ thay đổi một số cấu hình của ứng dụng trong tệp **build.gradle (Module: app**) để tìm hiểu cách thực hiện thay đổi và đồng bộ chúng với dự án Android Studio.

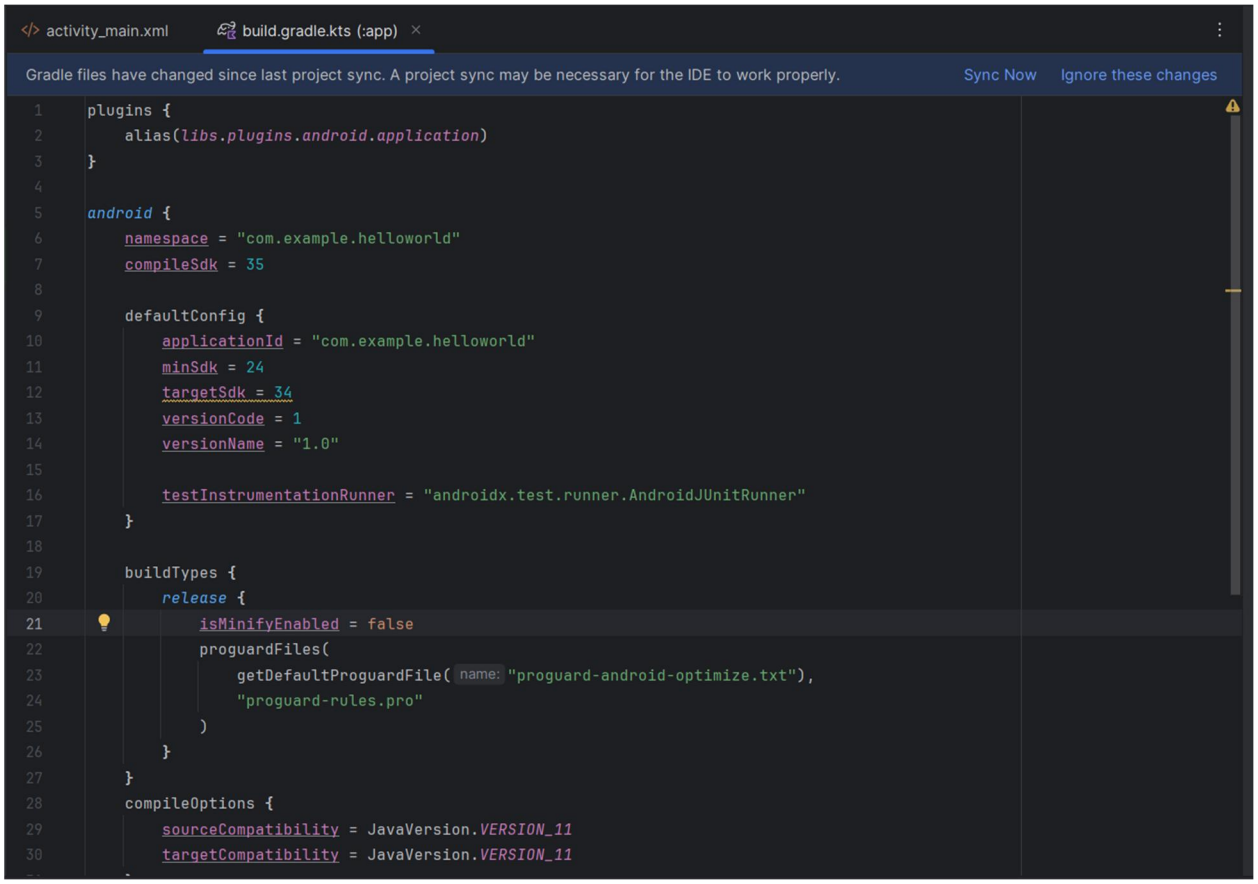
5.1. Thay đổi phiên bản SDK tối thiểu cho ứng dụng

Làm theo các bước sau :

1. Mở rộng thư **mục Gradle Scripts** nếu thư mục này chưa mở và nhấp đúp vào tệp **build.gradle(Module:app).**

   Nội dung của tệp sẽ xuất hiện trong trình chỉnh sửa mã.

2. Trong khối *defaultConfig*, hãy thay đổi giá trị của *minSdk* thành 24 như hiển thị bên dưới (ban đầu giá trị này được đặt thành 23).



   Trình chỉnh sửa mã hiển thị thanh thông báo ở trên cùng với liên kết **Sync Now** ngay.

5.2. Đồng bộ cấu hình Gradle mới

Khi bạn thực hiện thay đổi đối với các tệp cấu hình bản dựng trong một dự án, Android Studio yêu cầu bạn đồng bộ các tệp dự án để có thể nhập các thay đổi cấu hình bản dựng và chạy một số kiểm tra để đảm bảo cấu hình sẽ không tạo ra lỗi.

Để đồng bộ các tệp dự án, hãy nhấp vào **Sync Now** trên thanh thông báo xuất hiện khi thực hiện thay đổi (như được hiển thị trong hình trước) hoặc nhấp vào biểu tượng **Sync Project with Gradle Files** trên thanh công cụ.

Khi quá trình đồng bộ hóa Gradle hoàn tất, thông báo Gradle build finished sẽ xuất hiện ở góc dưới bên trái của cửa sổ Android Studio.

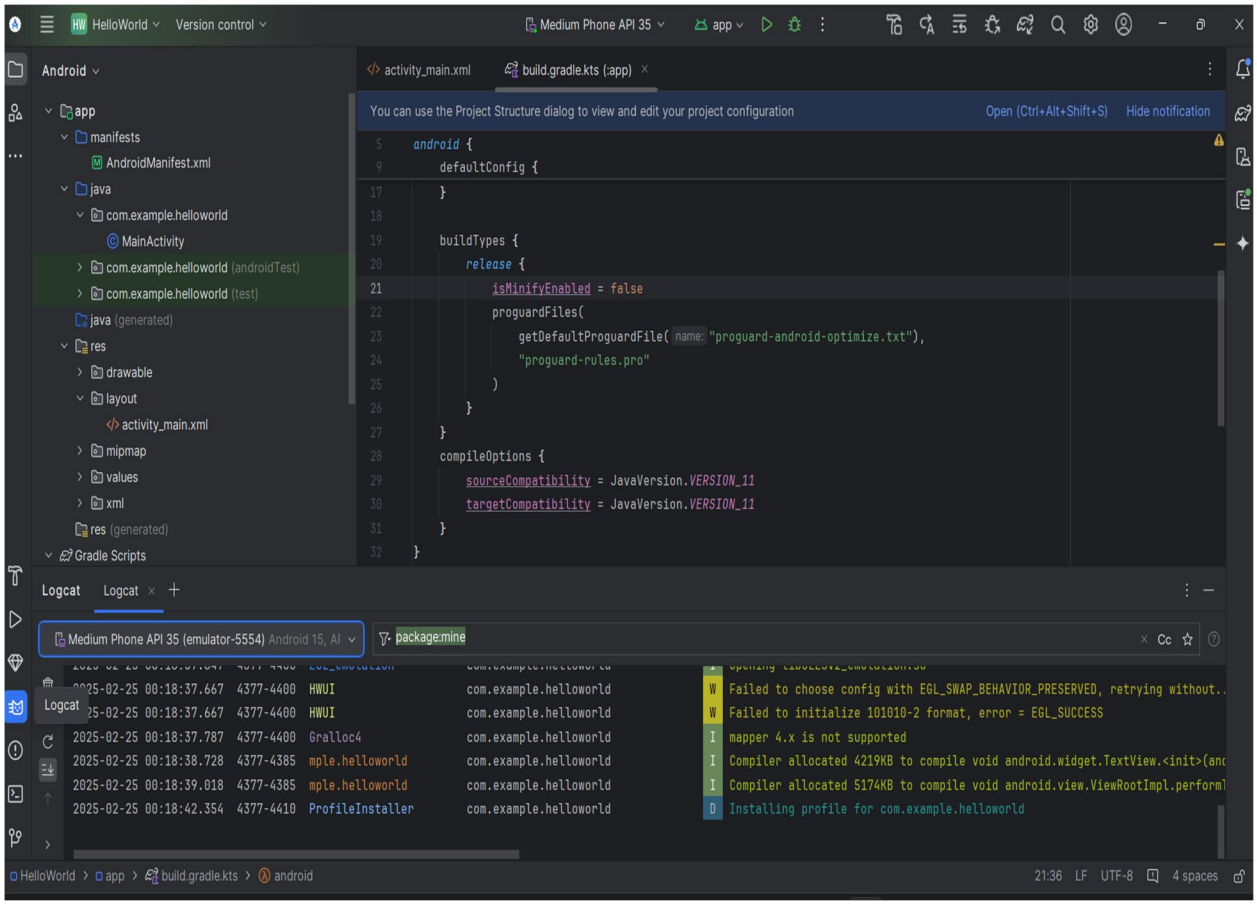
Để tìm hiểu sâu hơn về Gradle, hãy xem tài liệu Tổng quan về hệ thống bản dựng (Build System Overview) và Cấu hình bản dựng Gradle(Configuring Gradle Builds)

Nhiệm vụ 6: Thêm câu lệnh nhật ký vào ứng dụng của bạn

Trong nhiệm vụ này, bạn sẽ thêm các câu lệnh Log vào ứng dụng của mình, hiển thị các thông báo trong cửa sổ **Logcat**. Thông báo Log là một công cụ gỡ lỗi mạnh mẽ mà bạn có thể sử dụng để kiểm tra các giá trị, đường dẫn thực thi và báo cáo các ngoại lệ.

6.1. Cửa sổ Logcat

Để xem cửa sổ Logcat, hãy nhấp vào thẻ Logcat ở cuối cửa sổ Android Studio như được hiển thị trong hình bên dưới.



Trong hình trên:

1. Thẻ **Logcat** để mở và đóng cửa sổ **Logcat**, hiển thị thông tin về ứng dụng của bạn khi ứng dụng đang chạy. Nếu bạn thêm các câu lệnh Log vào ứng dụng, các thông báo Log sẽ xuất hiện ở đây.
2. Bộ menu cấp độ Log được đặt thành **Verbose** (mặc định), hiển thị tất cả các thông báo Log. Các cài đặt khác bao gồm **Debug**, **Error**, **Info** và **Warn**.

6.2. Thêm các câu lệnh nhật ký vào ứng dụng của bạn

Các câu lệnh nhật ký trong mã ứng dụng của bạn hiển thị thông báo trong ngăn Logcat. Ví dụ:

Log.d("MainActivity", "Hello World");

Các phần của thông báo là:

- Log: Lớp Log để gửi thông báo nhật ký đến cửa sổ Logcat.
- d: Cài đặt mức Nhật ký gỡ lỗi để lọc hiển thị thông báo nhật ký trong cửa sổ Logcat. Các mức nhật ký khác là **e** cho L**ỗi**, **w** cho **Cảnh báo** và **i** cho **Thông tin**.
- "MainActivity": Đối số đầu tiên là một thẻ có thể được sử dụng để lọc thông báo trong cửa sổ Logcat. Đây thường là tên của Hoạt động mà thông báo bắt nguồn. Tuy nhiên, bạn có thể biến điều này thành bất kỳ thứ gì hữu ích cho bạn để gỡ lỗi.

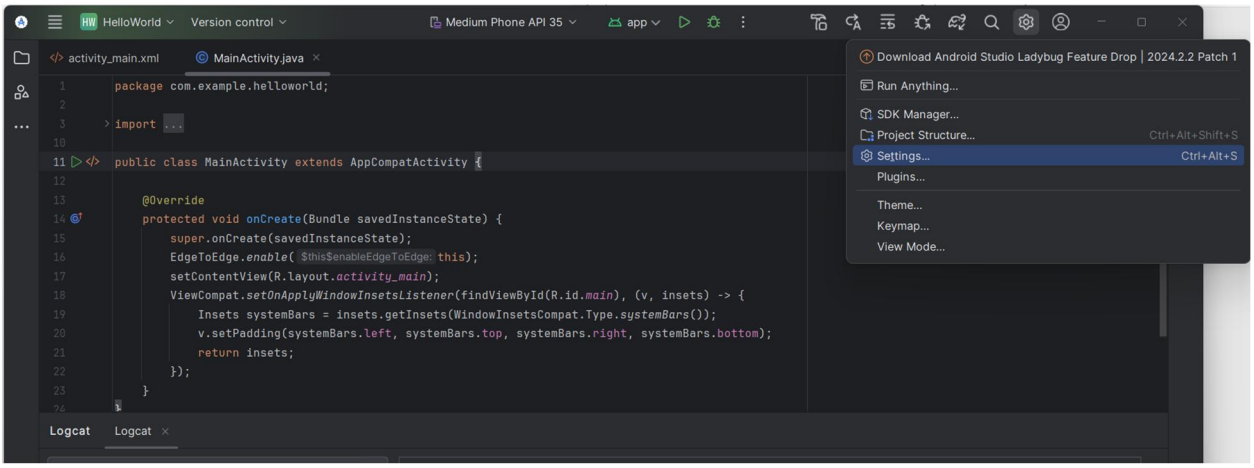
Theo quy ước, thẻ nhật ký được định nghĩa là hằng số cho hoạt động:

private static final String LOG_TAG = MainActivity.class.getSimpleName();

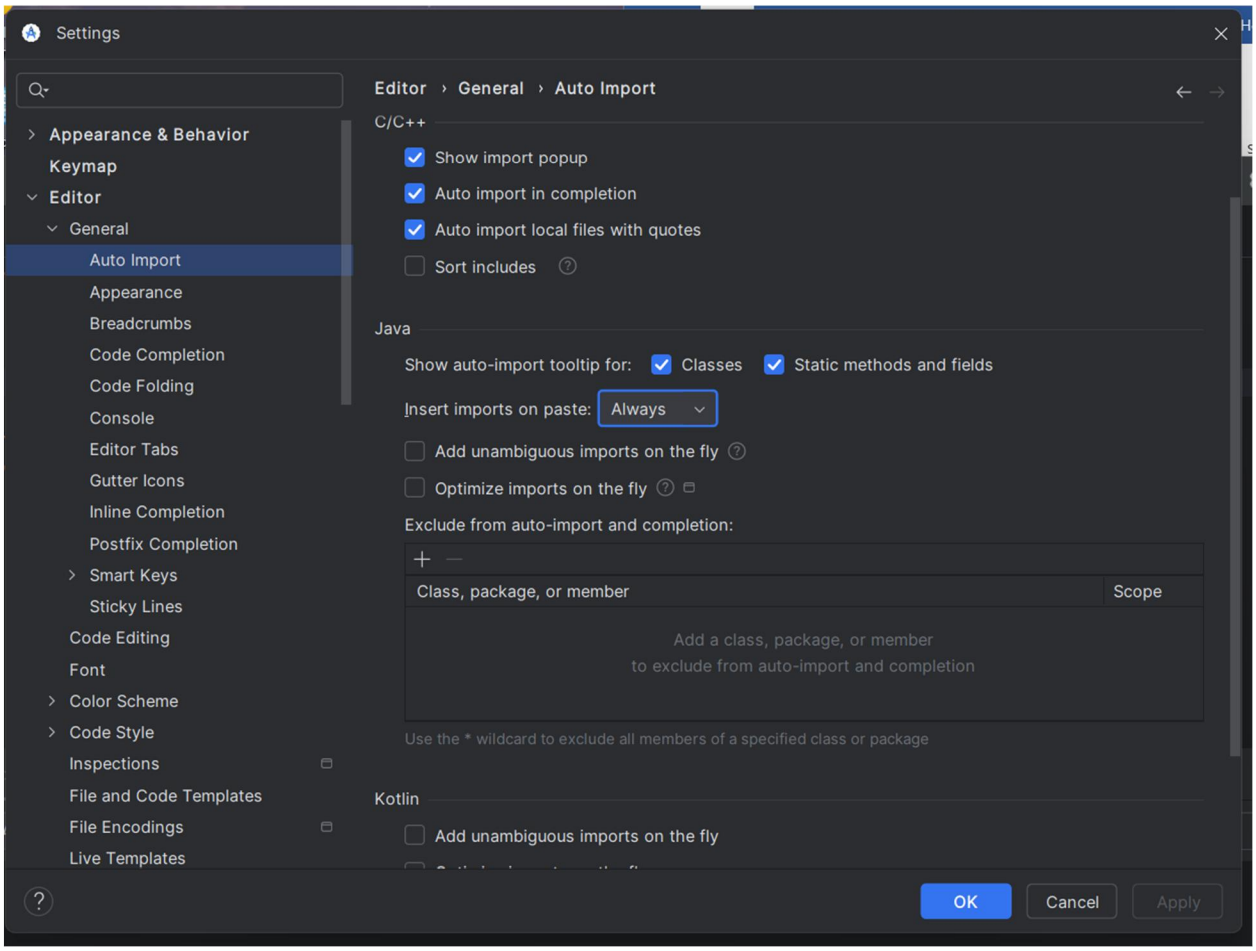
- " Hello world": Đối số thứ hai là thông điệp thực tế.

Thực hiện theo các bước sau:

1. Mở ứng dụng Hello World của bạn trong Android studio và mở MainActivity.
2. Để tự động thêm các mục nhập rõ ràng vào dự án của bạn (chẳng hạn như android.util.Log cần thiết để sử dụng Log), hãy chọn **File > Settings** trong Windows hoặc **Android Studio > Preferences** trong macOS.



3. Chọn **Editor > General >Auto Import**. Chọn tất cả các hộp và **đặt Insert imports on paste** thành **Always.**



4. Nhấp vào **Apply** rồi nhấp vào **OK**.
5. Trong phương thức onCreate() của MainActivity, hãy thêm câu lệnh sau:

Log.d("MainActivity", "Hello World");

Phương thức onCreate() bây giờ sẽ trông giống như đoạn mã sau:

```
package com.example.helloworld;

import android.os.Bundle;
import android.util.Log;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_main);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });
        Log.d( tag: "MainActivity",  msg: "Hello World");
    }
}
```

6. Nếu cửa sổ Logcat chưa mở, hãy nhấp vào thẻ **Logcat** ở cuối Android Studio để mở nó.
7. Kiểm tra xem tên mục tiêu và tên gói của ứng dụng có đúng không.
8. Thay đổi mức Nhật ký trong cửa sổ **Logcat** thành **Debug** (hoặc để nguyên là Verbose vì có quá ít thông báo nhật ký).
9. Chạy ứng dụng của bạn.

Thông báo sẽ xuất hiện ở cửa sổ Logcat:

```
2025-02-25 10:20:11.676  5034-5034  MainActivity          com.example.helloworld          D  Hello World
```

Thử thách mã hóa

Lưu ý: Tất cả các thử thách mã hóa đều là tùy chọn và không phải là điều kiện tiên quyết cho các bài học sau.

Thử thách: Bây giờ bạn đã thiết lập và quen thuộc với quy trình phát triển cơ bản, hãy thực hiện các bước sau:

1. Tạo một dự án mới trong Android Studio.
2. Đổi lời chào "Hello World" thành "Happy Birthday to " và tên của một người có sinh nhật gần đây.

3. (Tùy chọn) Chụp ảnh màn hình ứng dụng đã hoàn thành của bạn và gửi email cho một người có sinh nhật mà bạn đã quên.
4. Một cách sử dụng phổ biến của lớp Log là ghi lại các ngoại lệ Java khi chúng xảy ra trong chương trình của bạn. Có một số phương pháp hữu ích, chẳng hạn như Log.e(), mà bạn có thể sử dụng cho mục đích này. Khám phá phương pháp bạn có thể sử dụng để bao gồm ngoại lệ với thông báo Nhật ký. Sau đó, viết mã trong ứng dụng của bạn để kích hoạt và ghi nhật ký ngoại lệ.

Tóm tắt

- Để cài đặt Android Studio, truy cập Android Studio và làm theo hướng dẫn để tải xuống và cài đặt.
- Khi tạo ứng dụng mới, đảm bảo đặt **API 24: Android 7.0 Nougat** làm Minimum SDK.
- Để xem cấu trúc Android của ứng dụng trong **Project pane**, nhấp vào tab **Project** trong cột tab dọc, sau đó chọn **Android** trong menu bật lên ở trên cùng.
- Chỉnh sửa tệp **build.gradle(Module:app)** khi cần thêm thư viện mới hoặc thay đổi phiên bản thư viện.
- Tất cả mã nguồn và tài nguyên của ứng dụng nằm trong thư mục **app** và **res**. Thư mục java chứa các hoạt động, kiểm thử, và các thành phần mã Java khác. Thư mục res chứa tài nguyên như bố cục giao diện, chuỗi văn bản, hình ảnh.
- Chỉnh sửa tệp AndroidManifest.xml để thêm các thành phần và quyền cho ứng dụng Android.
- Sử dụng Android Virtual Device (AVD) Manager để tạo thiết bị ảo (emulator) chạy ứng dụng.
- Thêm câu lệnh Log vào ứng dụng để hiển thị thông điệp trong **Logcat pane** nhằm hỗ trợ gỡ lỗi.
- Để chạy ứng dụng trên thiết bị Android thật, bật **USB Debugging** trên thiết bị. Mở **Settings > About phone**, nhấn vào **Build number** bảy lần. Quay lại **Settings** màn hình, vào **Developer options**, bật **USB Debugging**.

Các khái niệm liên quan

Tài liệu về khái niệm liên quan có trong phiên bản 1.0: Giới thiệu về Android và phiên bản 1.1 Ứng dụng Android đầu tiên của bạn.

Tìm hiểu thêm

- Tài liệu Android Studio:
- Trang tải xuống Android Studio
- Ghi chú phát hành Android Studio
- Làm quen với Android Studio
- Công cụ dòng lệnh Logcat
- Trình quản lý Thiết bị ảo Android (AVD)
- Tổng quan về App Manifest
- Cấu hình bản dựng của bạn
- Lớp nhật ký
- Tạo và quản lý Thiết bị ảo

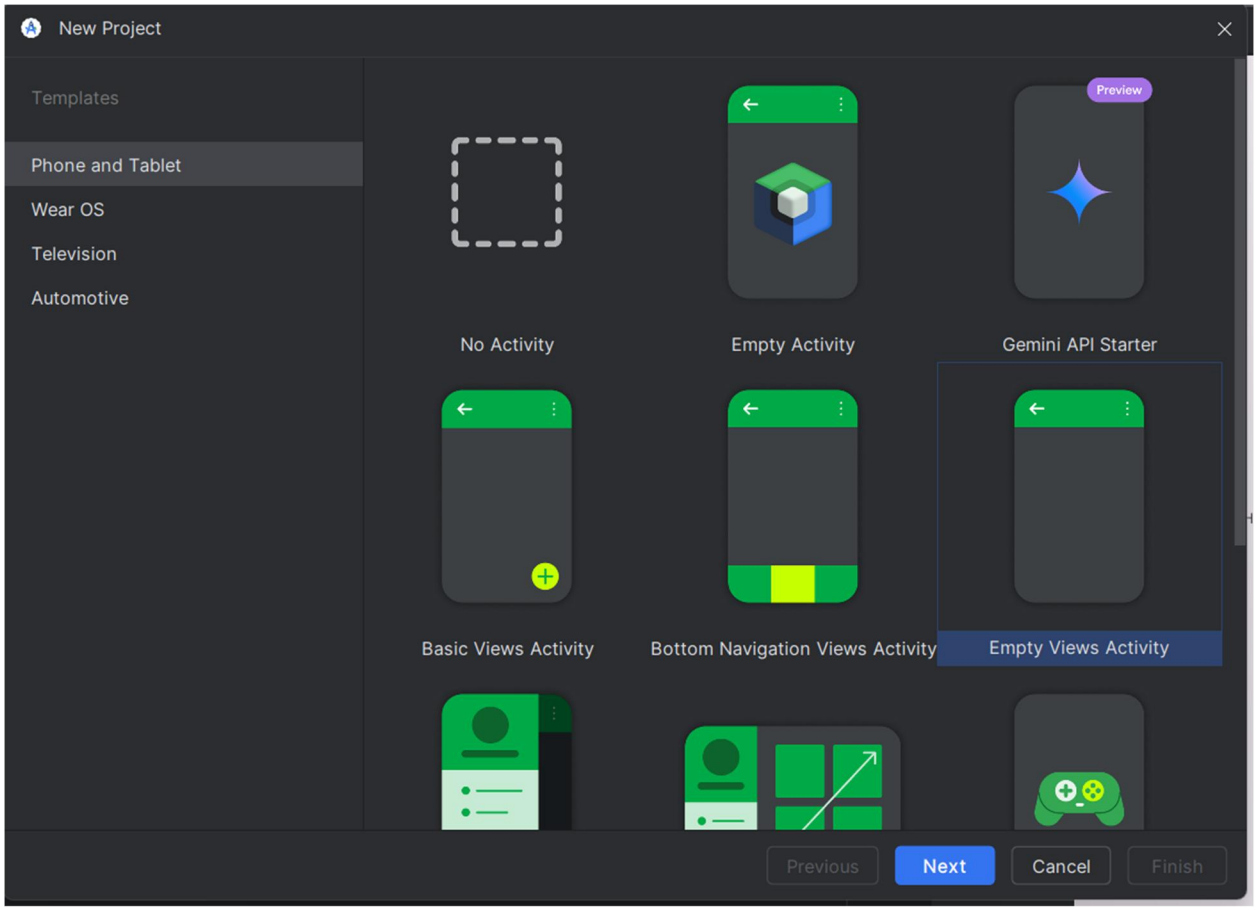
Khác:

- Làm thế nào để cài đặt Java?
- Cài đặt phần mềm JDK và thiết lập JAVA_HOME
- Trang Gradle
- Cú pháp Apache Groovy
- Trang Wikipedia Gradle

Bài tập

Xây dựng và chạy ứng dụng

- Tạo một dự án Android mới từ mẫu.

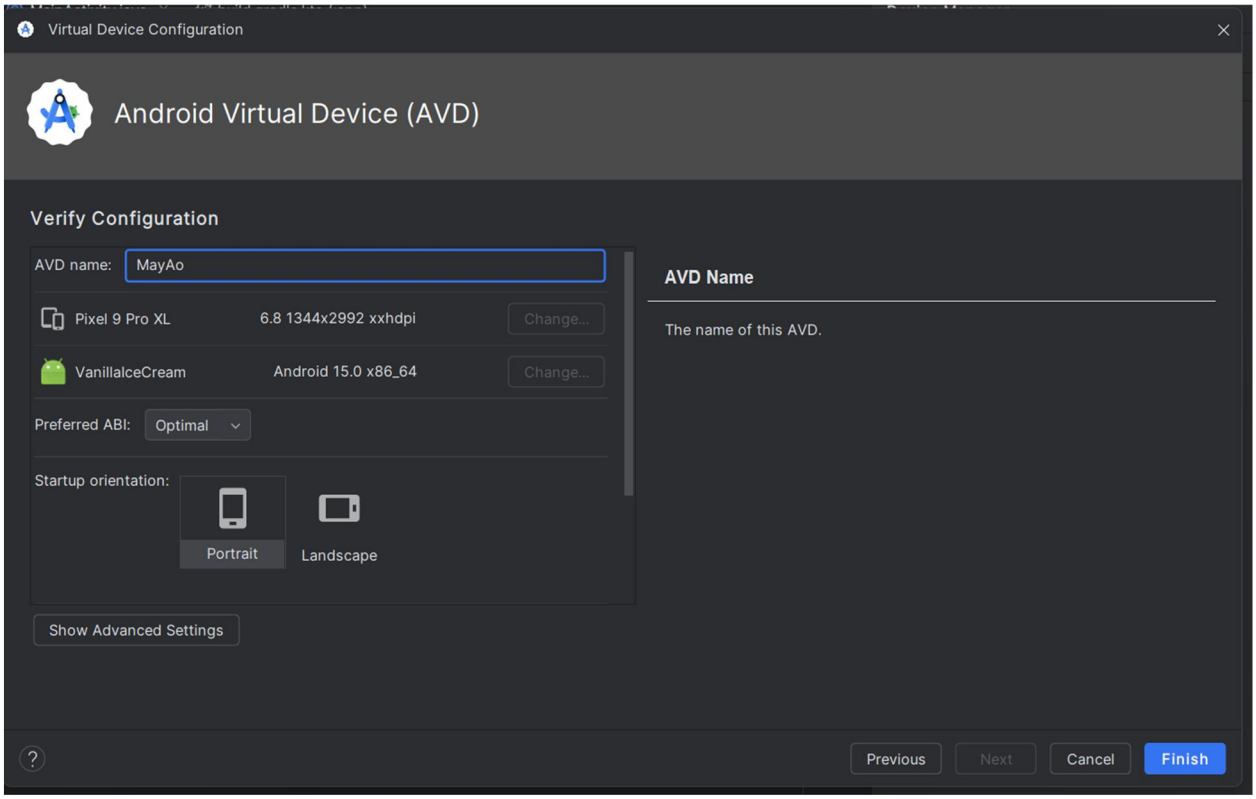


- Thêm các câu lệnh ghi nhật ký cho nhiều cấp độ nhật ký khác nhau trong onCreate() trong hoạt động chính.

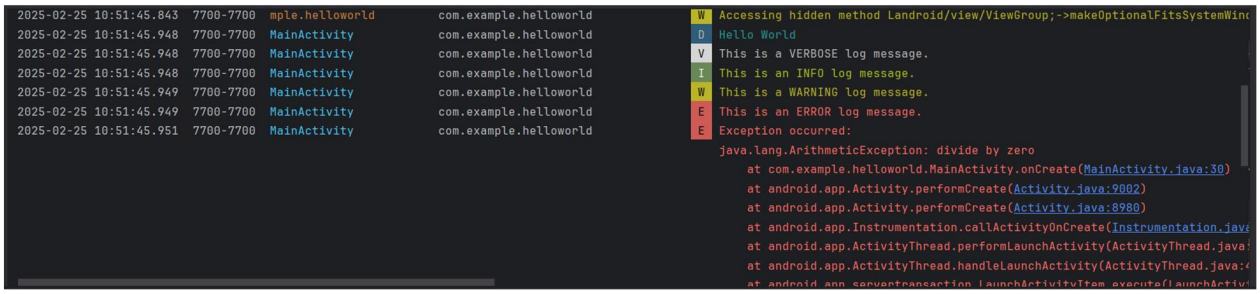
activity_main.xml | MainActivity.java | build.gradle.kts (:app)

```java
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_main);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });
        Log.d( tag: "MainActivity",  msg: "Hello World");
        Log.v( tag: "MainActivity",  msg: "This is a VERBOSE log message.");
        Log.i( tag: "MainActivity",  msg: "This is an INFO log message.");
        Log.w( tag: "MainActivity",  msg: "This is a WARNING log message.");
        Log.e( tag: "MainActivity",  msg: "This is an ERROR log message.");
        try {
            int result = 10 / 0;
        } catch (Exception e) {
            Log.e( tag: "MainActivity",  msg: "Exception occurred: ", e);
        }
    }
}
```

- Tạo trình giả lập cho thiết bị, nhắm mục tiêu đến bất kỳ phiên bản Android nào bạn thích và chạy ứng dụng.



- Sử dụng tính năng lọc trong **Logcat** để tìm các câu lệnh nhật ký của bạn và điều chỉnh các cấp độ để chỉ hiển thị các câu lệnh ghi nhật ký gỡ lỗi hoặc lỗi.



Trả lời các câu hỏi sau

Câu hỏi 1: Tên của tệp bố cục cho hoạt động chính là gì?

- MainActivity.java
- AndroidManifest.xml
- activity_main.xml
- build.gradle

Câu hỏi 2:Tên của tài nguyên chuỗi chỉ định tên ứng dụng là gì?

- app_name
- xmlns:app
- android:name
- applicationId

Câu hỏi 3: Bạn sử dụng công cụ nào để tạo trình giả lập mới?

- Android Device Monitor
- AVD Manager
- SDK Manager
- Theme Editor

Câu hỏi 4: Giả sử ứng dụng của bạn bao gồm câu lệnh ghi nhật ký này:

Log.i("MainActivity", "MainActivity layout is complete");

Bạn thấy câu lệnh "MainActivity layout is complete" trong ngăn Logcat nếu menu Log level được đặt thành tùy chọn nào sau đây? (Gợi ý: nhiều câu trả lời là được.)

- Verbose (Chi tiết)
- Debug (Gỡ lỗi)
- Info (Thông tin)
- Warn (Cảnh báo)
- Error (Lỗi)
- Assert (Khẳng định)

Nộp ứng dụng của bạn để chấm điểm

Kiểm tra để đảm bảo ứng dụng có những thông tin sau:

- Hoạt động hiển thị "Hello World" trên màn hình.
- Câu lệnh log trong onCreate() trong hoạt động chính.
- Mức log trong ngăn Logcat chỉ hiển thị câu lệnh ghi nhật ký gỡ lỗi hoặc lỗi.

1.2) Giao diện người dùng tương tác đầu tiên

Giới thiệu

Giao diện người dùng (UI) xuất hiện trên màn hình của một thiết bị Android bao gồm một hệ thống phân cấp các đối tượng được gọi là views — mỗi phần tử trên màn hình đều là một View. Lớp View đại diện cho khối xây dựng cơ bản của tất cả các thành phần UI và là lớp cơ sở cho các lớp cung cấp các thành phần UI tương tác, chẳng hạn như nút bấm (Button), hộp kiểm (Checkbox), và trường nhập văn bản (Text Entry Field). Các lớp con của View được sử dụng phổ biến và sẽ được mô tả qua nhiều bài học bao gồm:

- TextView để hiển thị văn bản.
- EditText để cho phép người dùng nhập và chỉnh sửa văn bản.
- Button và các thành phần có thể nhấp khác (như RadioButton, CheckBox và Spinner) để cung cấp hành vi tương tác.
- ScrollView và RecyclerView để hiển thị các mục có thể cuộn.
- ImageView để hiển thị hình ảnh.
- ConstraintLayout và LinearLayout để chứa các thành phần View khác và định vị chúng.

Mã Java hiển thị và điều khiển UI được chứa trong một lớp mở rộng Activity. Activity thường được liên kết với bố cục của các chế độ xem UI được xác định là tệp XML (Ngôn ngữ đánh dấu mở rộng). Tệp XML này thường được đặt tên theo Activity của nó và xác định bố cục của các thành phần View trên màn hình.

Ví dụ, mã MainActivity trong ứng dụng Hello World hiển thị bố cục được xác định trong tệp bố cục activity_main.xml, bao gồm TextView có văn bản "Hello World".

Trong các ứng dụng phức tạp hơn, một Hoạt động có thể triển khai các hành động để phản hồi các lần chạm của người dùng, vẽ nội dung đồ họa hoặc yêu cầu dữ liệu từ cơ sở dữ liệu hoặc internet. Bạn sẽ tìm hiểu thêm về lớp Hoạt động trong một bài học khác.

Trong bài thực hành này, bạn sẽ học cách tạo ứng dụng tương tác đầu tiên của mình- một ứng dụng cho phép người dung tương tác. Bạn sẽ tạo một ứng dụng bằng mẫu Hoạt động trống. Bạn cũng sẽ học cách sử dụng trình chỉnh sửa bố cục để thiết kế bố cục và cách chỉnh sửa bố cục trong XML. Bạn cần phát triển các kỹ năng này để có thể hoàn thành các bài thực hành khác trong khóa học này.

Những điều bạn nên biết

Bạn nên quen thuộc với:

- Cách cài đặt và mở Android Studio.

- Cách tạo ứng dụng HelloWorld.
- Cách chạy ứng dụng HelloWorld.

Những thứ bạn sẽ tìm hiểu

- Cách tạo ứng dụng có hành vi tương tác.
- Cách sử dụng trình chỉnh sửa bố cục để thiết kế bố cục.
- Cách chỉnh sửa bố cục trong XML.
- Nhiều thuật ngữ mới. Hãy xem qua từ vựng và khái niệm trong bảng chú giải thuật ngữ để có các định nghĩa thân thiện

Những thứ bạn sẽ làm

- Tạo một ứng dụng và thêm hai phần tử Button và một TextView vào bố cục.
- Thao tác từng phần tử trong ConstraintLayout để giới hạn chúng vào lề và các phần tử khác.
- Thay đổi các thuộc tính phần tử UI.
- Chỉnh sửa bố cục của ứng dụng trong XML.
- Trích xuất các chuỗi được mã hóa cứng thành các tài nguyên chuỗi.
- Triển khai các phương thức xử lý nhấp để hiển thị thông báo trên màn hình khi người dùng chạm vào mỗi Button.

Tổng quan về ứng dụng

Ứng dụng HelloToast bao gồm hai Button và một TextView. Khi người dùng nhấn vào Nút đầu tiên nó hiển thị một thông báo ngắn (**Toast**) trên màn hình. Nhấn nút thứ hai Tăng giá trị bộ đếm số lần nhấn và hiển thị trong TextView, bắt đầu từ 0.

Dưới đây là giao diện của ứng dụng đã hoàn thiện:

Nhiệm vụ 1: Tạo và khám phá một dự án mới

Trong bài thực hành này, bạn thiết kế và triển khai một dự án cho ứng dụng HelloToast. Một liên kết đến mã giải pháp được cung cấp ở cuối.

1.1 Tạo dự án Android Studio

1. Khởi động Android Studio và tạo một dự án mới với các tham số sau:

Giá trị thuộc tính	Giá trị
Tên ứng dụng Hello Toast	**Hello Toast**

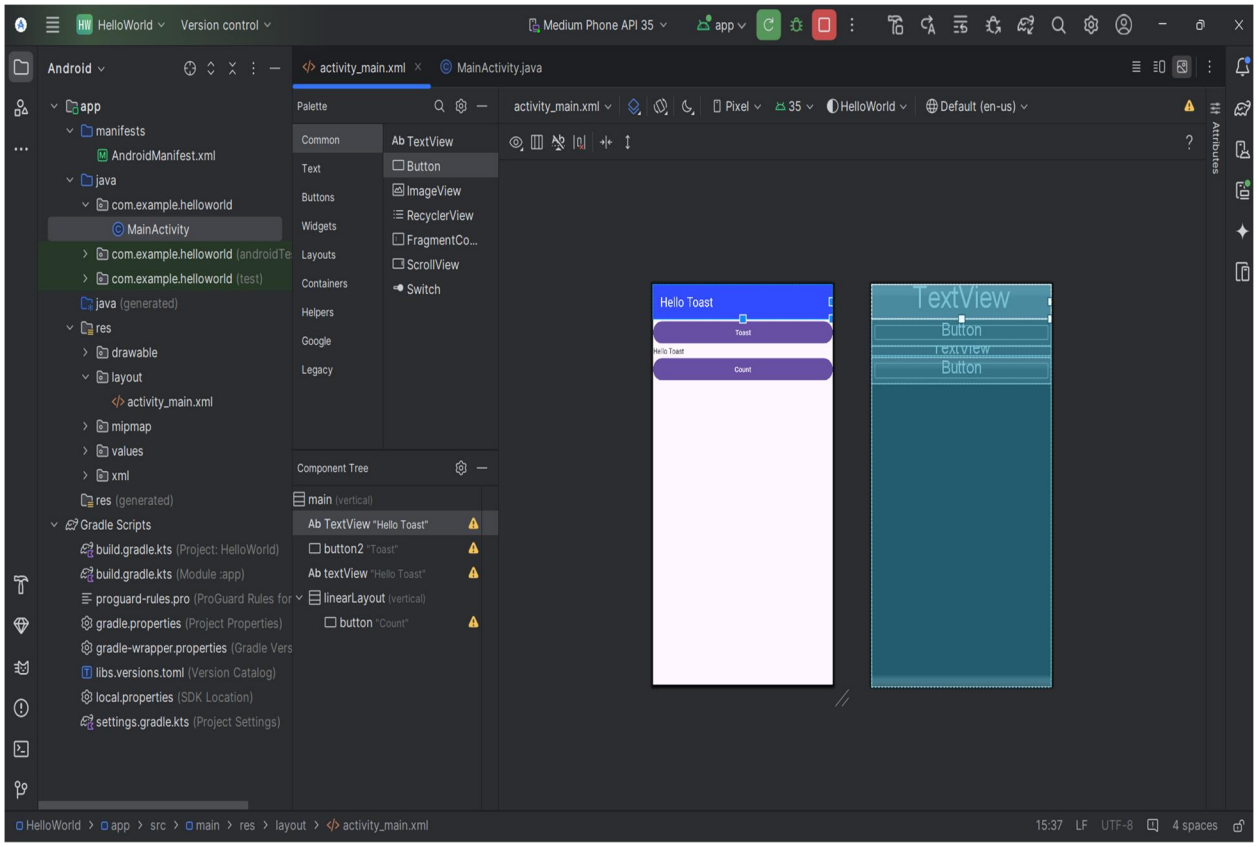
Tên công ty	**com.example.android** (hoặc tên miền của riêng bạn)
Điện thoại và Máy tính bảng SDK tối thiểu	**API 24: Android 7.0 Nougat**
Mẫu	**Empty View Activity**
Hộp Tạo tệp Bố cục	Đã chọn
Hộp Tương Thích Ngược	Đã chọn

2. Chọn **Run > Run app** hoặc nhấp vào **biểu tượng Run** trên thanh công cụ để biên dịch và chạy ứng dụng trên trình giả lập hoặc thiết bị của bạn.

1.2 Khám phá trình chỉnh sửa bố cục (Layout Editor)

Android Studio cung cấp trình chỉnh sửa bố cục để nhanh chóng xây dựng bố cục của các thành phần giao diện người dùng (UI) của ứng dụng. Nó cho phép bạn kéo các thành phần vào chế độ xem thiết kế trực quan và bản thiết kế, định vị chúng trong bố cục, thêm ràng buộc và đặt thuộc tính. Ràng buộc xác định vị trí của thành phần UI trong bố cục. Ràng buộc thể hiện kết nối hoặc căn chỉnh với chế độ xem khác, bố cục cha hoặc hướng dẫn vô hình.

Khám phá trình chỉnh sửa bố cục và tham khảo hình bên dưới khi bạn làm theo các bước được đánh số:



1. Trong thư mục **app > res > layout** trong ngăn **Project > Android**, hãy nhấp đúp vào tệp **activity_main.xml** để mở tệp đó, nếu tệp đó chưa được mở.
2. Nhấp vào tab **Design** nếu tệp đó chưa được chọn. Bạn sử dụng tab **Design** để thao tác các phần tử và bố cục, và thẻ **Text** để chỉnh sửa mã XML cho bố cục.
3. Cửa sổ **Palettes** hiển thị các phần tử UI mà bạn có thể sử dụng trong bố cục của ứng dụng.
4. Ngăn **Component tree** hiển thị hệ thống phân cấp chế độ xem của các phần tử UI. Các phần tử View được tổ chức thành hệ thống phân cấp cây gồm các phần tử cha và con, trong đó phần tử con kế thừa các thuộc tính của phần tử cha. Trong hình trên, TextView là phần tử con của ConstraintLayout. Bạn sẽ tìm hiểu về các phần tử này sau trong bài học này.
5. Ngăn thiết kế và bản thiết kế của trình chỉnh sửa bố cục hiển thị các phần tử UI trong bố cục. Trong hình trên, bố cục chỉ hiển thị một phần tử: TextView hiển thị "Hello World".
6. Thẻ **Attributes** hiển thị ngăn **Attributes** để thiết lập thuộc tính cho phần tử UI.

Mẹo: Xem Xây dựng giao diện người dùng bằng Trình chỉnh sửa bố cục để biết chi tiết về cách sử dụng trình chỉnh sửa bố cục và Meet Android Studio để biết tài liệu đầy đủ về Android Studio.

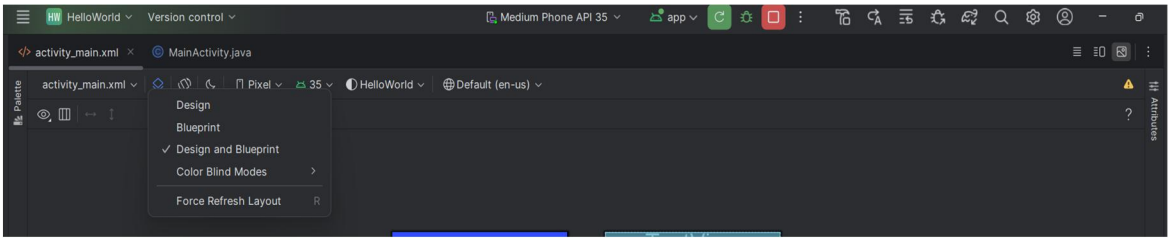
Nhiệm vụ 2: Thêm các phần tử View trong Layout Editor

Trong phần này, bạn sẽ tạo bố cục UI cho ứng dụng HelloToast bằng cách sử dụng ConstraintLayout. Bạn có thể thiết lập ràng buộc thủ công hoặc tự động bằng công cụ **Autoconnect**.

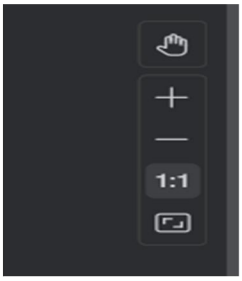
2.1 Kiểm tra ràng buộc của phần tử UI

Làm theo các bước sau:

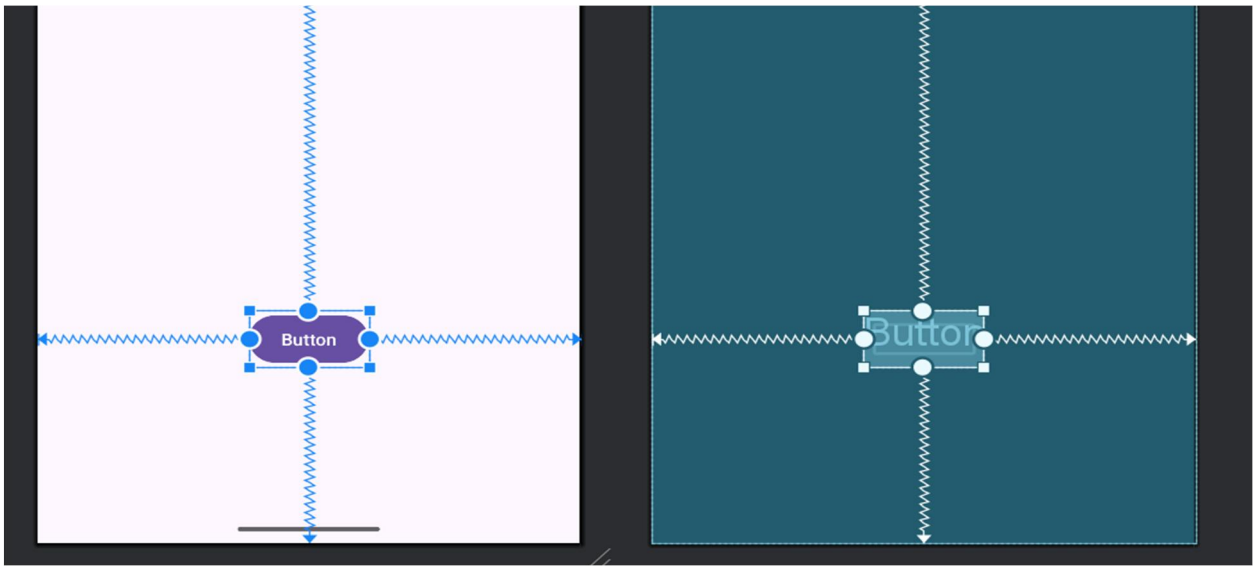
1. Mở *activity_main.xml* từ ngăn **Project > Android** nếu nó chưa mở. Nếu thẻ **Design** chưa được chọn, hãy nhấp vào tab đó.
   Nếu không có bản thiết kế, hãy nhấp vào nút **Select Design Surface** trên thanh công cụ và chọn **Design and Blueprint.**



2. Công cụ **Autoconnect** cũng nằm trên thanh công cụ. Theo mặc định, công cụ này được bật. Đối với bước này, hãy đảm bảo rằng công cụ không bị tắt.
3. Nhấp vào nút phóng to để phóng to các ngăn thiết kế và bản thiết kế để xem cận cảnh.



4. Chọn **TextView** trong ngăn Component Tree. TextView "Hello World" được tô sáng trong các ngăn thiết kế và bản thiết kế và các ràng buộc cho phần tử có thể nhìn thấy được.
5. Tham khảo hình ảnh động bên dưới để biết bước này. Nhấp vào tay cầm hình tròn ở bên phải của TextView để xóa ràng buộc theo chiều ngang liên kết chế độ xem với bên phải của bố cục. TextView nhảy sang bên trái vì nó không còn bị ràng buộc ở bên phải nữa. Để thêm lại ràng buộc theo chiều ngang, hãy nhấp vào tay cầm tương tự và kéo một đường sang bên phải của bố cục.



Trong bản thiết kế hoặc ngăn thiết kế, các nút điều khiển sau xuất hiện trên phần tử TextView:

- **Constraint handle**: Để tạo một ràng buộc như trong hình động ở trên, hãy nhấp vào một constraint handle, được hiển thị dưới dạng một vòng tròn ở bên cạnh một phần tử. Sau đó, kéo handle đến một constraints khác hoặc đến ranh giới parent. Một đường ngoằn ngoèo biểu thị cho constraints.

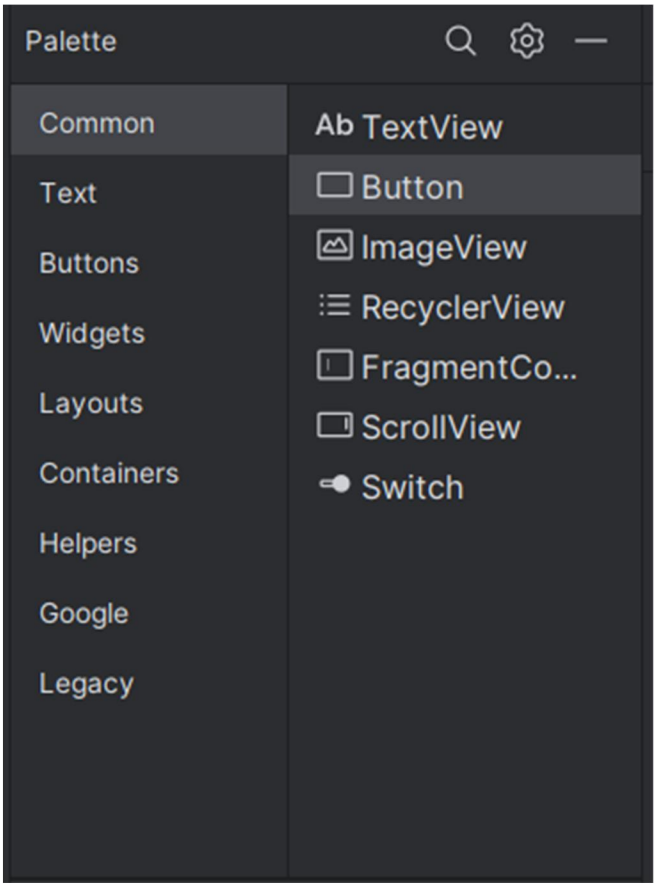
- **Resizing handle**: Để thay đổi kích thước phần tử, hãy kéo các handle thay đổi kích thước hình vuông. Handle sẽ thay đổi thành một góc nghiêng khi bạn kéo nó.

2.2 Thêm 1 nút vào bố cục

Khi được bật, công cụ Autoconnect sẽ tự động tạo hai hoặc nhiều ràng buộc cho một thành phần UI vào bố cục cha. Sau khi bạn kéo thành phần vào bố cục, công cụ này sẽ tạo ra các ràng buộc dựa trên vị trí của thành phần.

Thực hiện theo các bước sau để thêm Nút:

1. Bắt đầu với một bảng trắng. Thành phần TextView không cần thiết, vì vậy khi thành phần này vẫn được chọn,hãy nhấn phím **Delete** hoặc chọn **Edit > Delete**. Bây giờ bạn có một trang bố cục hoàn toàn trống.
2. Kéo **Button** từ ngăn Palette đến bất kỳ vị trí nào trong bố cục. Nếu bạn thả Nút vào khu vực giữa trên cùng của bố cục, các ràng buộc có thể tự động xuất hiện. Nếu không, bạn có thể kéo các ràng buộc lên trên cùng, bên trái và bên phải của bố cục như minh họa trong hình động bên dưới.



2.3 Thêm nút thứ hai vào bố cục

1. Kéo **Button** khác từ ngăn **Palette** màu vào giữa bố cục như thể hiện trong hình động bên dưới. Autoconnect có thể cung cấp các ràng buộc theo chiều ngang cho bạn (nếu không, bạn có thể tự kéo chúng).
2. Kéo một ràng buộc theo chiều dọc xuống dưới cùng của bố cục (tham khảo hình bên dưới).

1.3) Trình chỉnh sửa bố cục

1.4) Văn bản và các chế độ cuộn

1.5) Tài nguyên có sẵn

Bài 2) Activities

2.1) Activity và Intent

2.2) Vòng đời của Activity và trạng thái

2.3) Intent ngầm định

Bài 3) Kiểm thử, gỡ lỗi và sử dụng thư viện hỗ trợ

3.1) Trình gỡ lỗi

3.2) Kiểm thử đơn vị

3.3) Thư viện hỗ trợ

CHƯƠNG 2. TRẢI NGHIỆM NGƯỜI DÙNG

Bài 1) Tương tác người dùng

1.1) Hình ảnh có thể chọn

1.2) Các điều khiển nhập liệu

1.3) Menu và bộ chọn

1.4) Điều hướng người dùng

1.5) RecycleView

Bài 2) Trải nghiệm người dùng thú vị

2.1) Hình vẽ, định kiểu và chủ đề

2.2) Thẻ và màu sắc

2.3) Bố cục thích ứng

Bài 3) Kiểm thử giao diện người dùng

3.1) Espresso cho việc kiểm tra UI

CHƯƠNG 3. LÀM VIỆC TRONG NỀN

Bài 1) Các tác vụ nền

1.1) AsyncTask

1.2) AsyncTask và AsyncTaskLoader

1.3) Broadcast receivers

Bài 2) Kích hoạt, lập lịch và tối ưu hóa nhiệm vụ nền

2.1) Thông báo

2.2) Trình quản lý cảnh báo

2.3) JobScheduler

CHƯƠNG 4. LƯU DỮ LIỆU NGƯỜI DÙNG

Bài 1) Tùy chọn và cài đặt

1.1) Shared preferences

1.2) Cài đặt ứng dụng

Bài 2) Lưu trữ dữ liệu với Room

2.1) Room, LiveData và ViewModel

2.2) Room, LiveData và ViewModel